

AIDOS LAB
AI FOR DATA-ORIENTED SCIENCE

UNI FR

Lecture 9: topological losses

Geometry and Topology in Machine Learning

Inés García Redondo — November 10, 2026

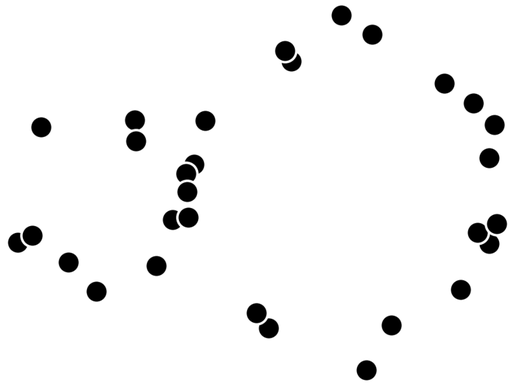
Recap

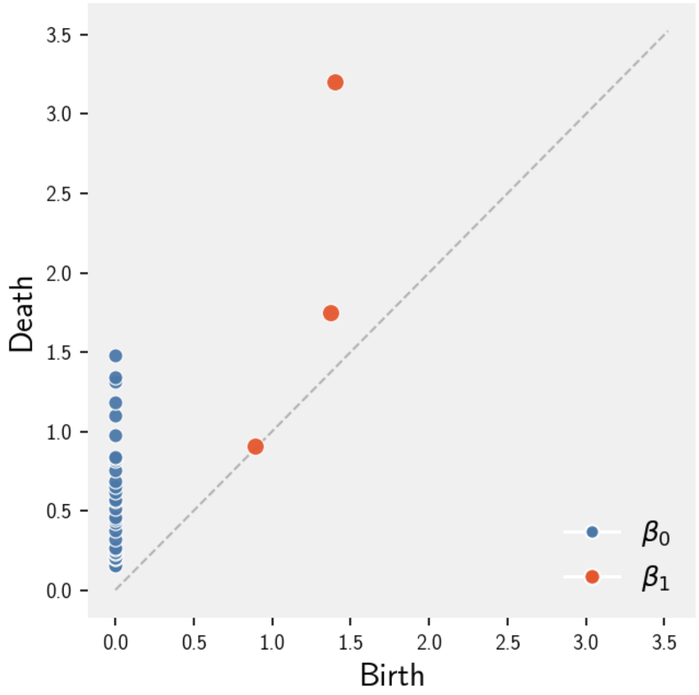
Lecture 9 — Topological losses

Persistent homology pipeline



Point cloud ⟶ Čech ⟶ Persistence diagram ⟶ ???

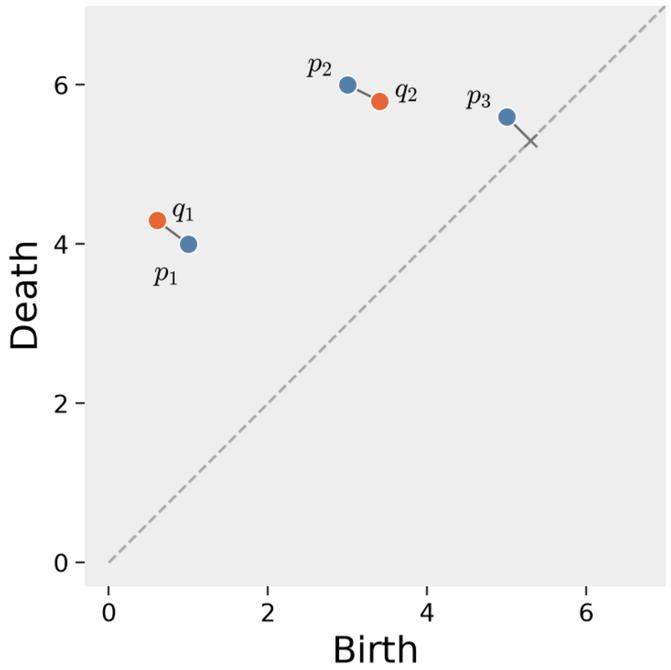




Distances between diagrams

Def. Matching between persistence diagrams

A matching between persistence diagrams is a bijection $\psi : \mathrm{PD}(X) \to \mathrm{PD}(Y)$ between the points in the diagrams, where the points in the diagonal in both diagrams are counted with infinite multiplicity



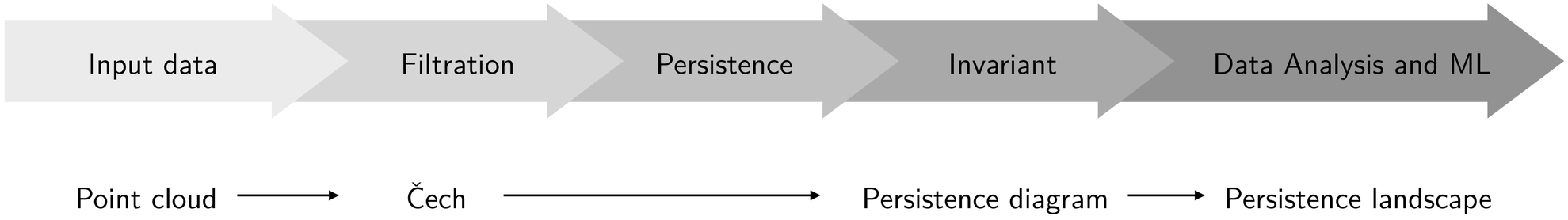
Bottleneck distance

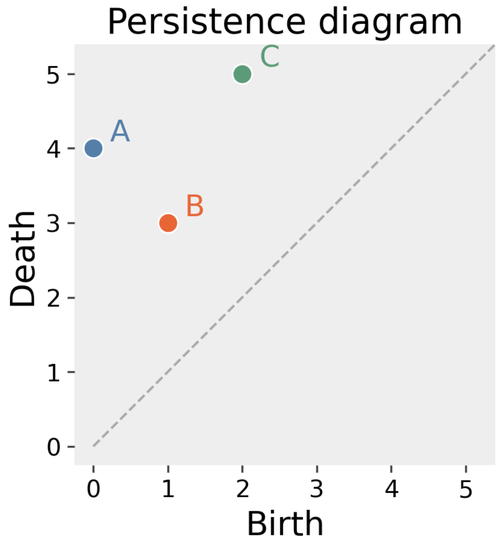
$$d_B\left(\mathrm{PD}(X), \mathrm{PD}(Y)\right) = \inf_{\psi:\mathrm{PD}(X)\to\mathrm{PD}(Y)} \max\{\|x - \psi(x)\|_\infty : \forall x \in \mathrm{PD}(X)\}$$

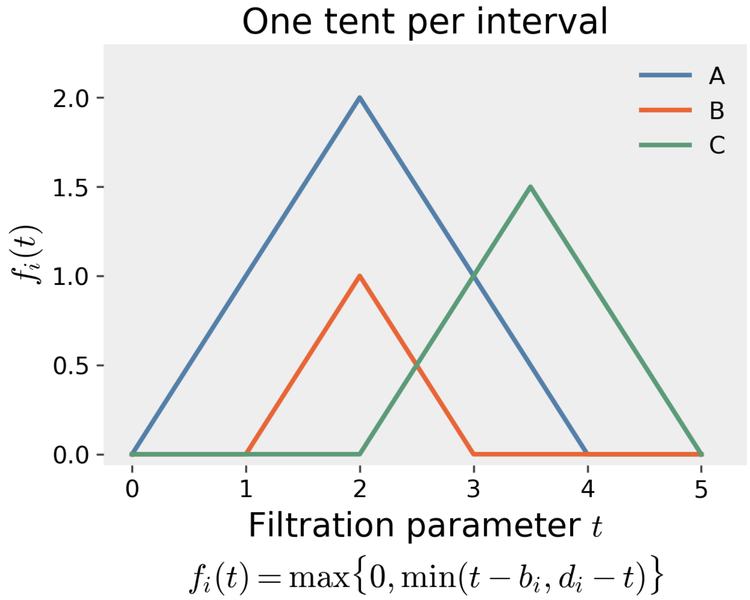
Wasserstein distance

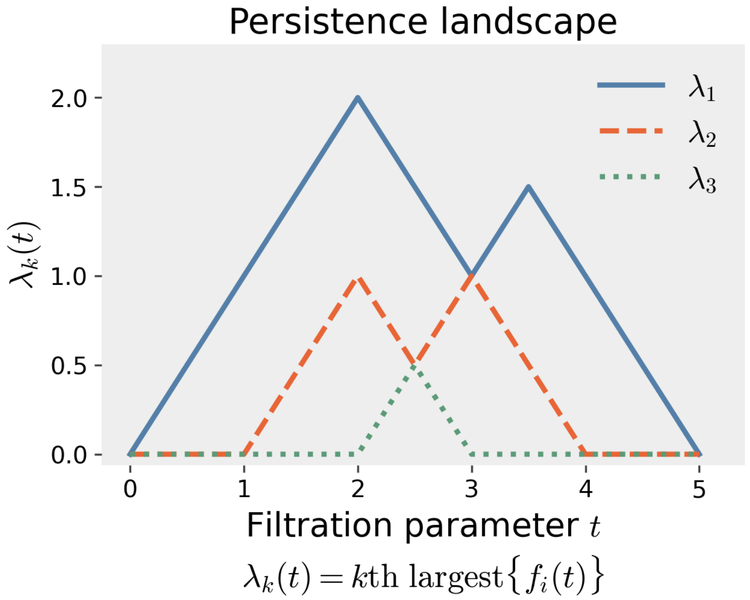
$$W_p\left(\mathrm{PD}(X), \mathrm{PD}(Y)\right) = \inf_{\psi:\mathrm{PD}(X)\to\mathrm{PD}(Y)} \left(\sum_{x\in\mathrm{PD}(X)} \|x - \psi(x)\|_p^p \right)^{1/p}$$

Persistent homology pipeline









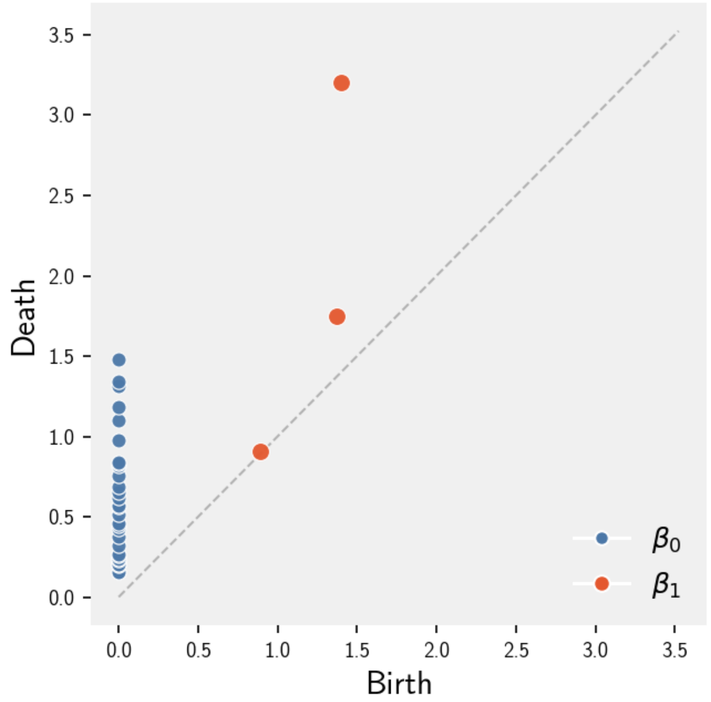
PersLay: learning from persistence diagrams

Lecture 9 — Topological losses

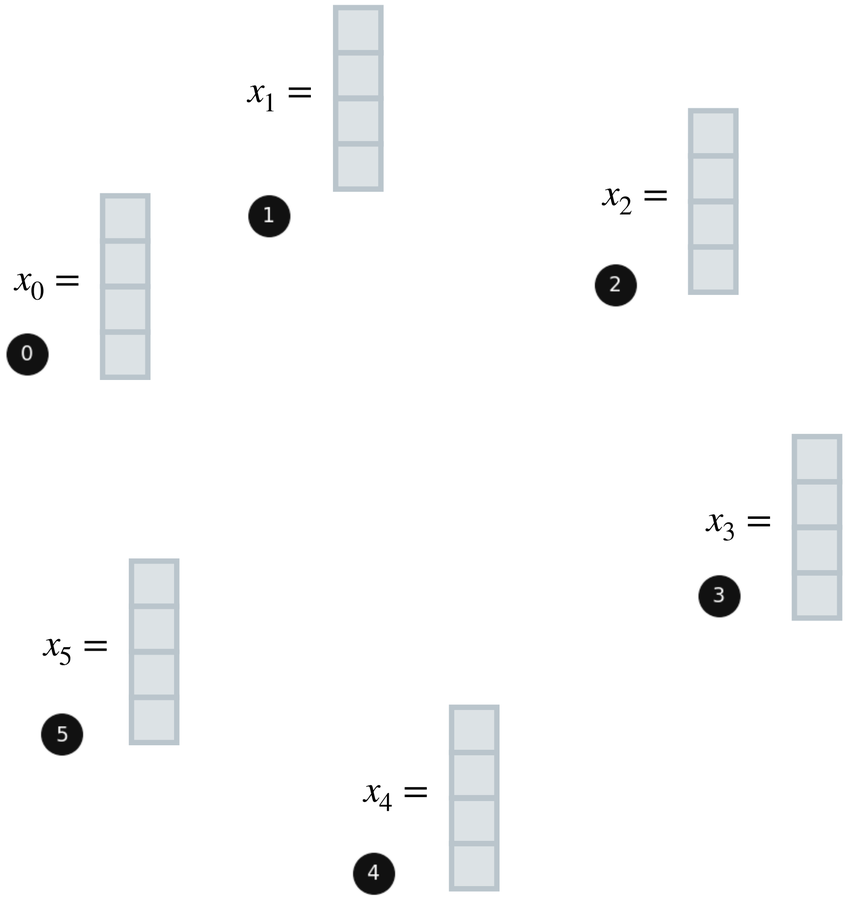
Persistent homology pipeline



Point cloud ⟶ Čech ⟶ Persistence diagram ⟶ ???



Learning from sets

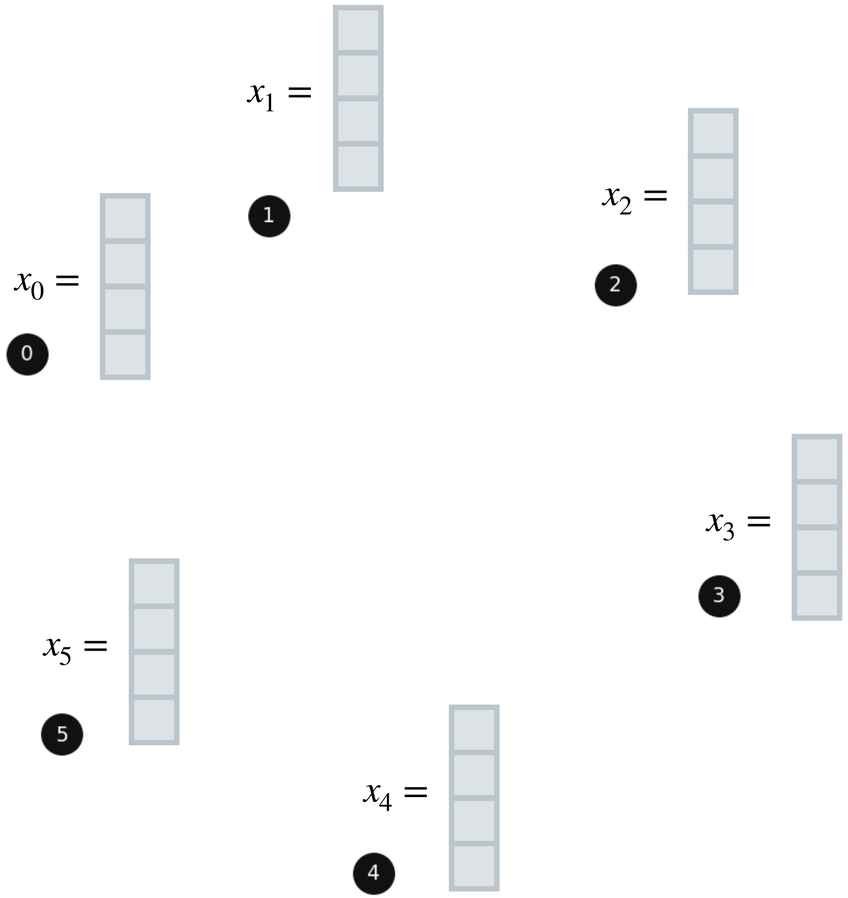


- Suppose we are given a set of points S, and we want to learn a function $f : S \to \mathbb{R}^k$
- In other words, we want to learn feature vectors on the points of the set: $x_i \in \mathbb{R}^d$ for $i \in S$
- We stack them: $X = (x_1, \ldots, x_n)^\top \in \mathbb{R}^{n \times d}$

  Already specifies an order in which you are visiting the nodes

  But our function should not depend on this choice!

Learning from sets



What transformations change the ordering of the nodes?

Permutations $\sigma \in S_n$

Can be represented as *permutation matrices*, which we can apply to X

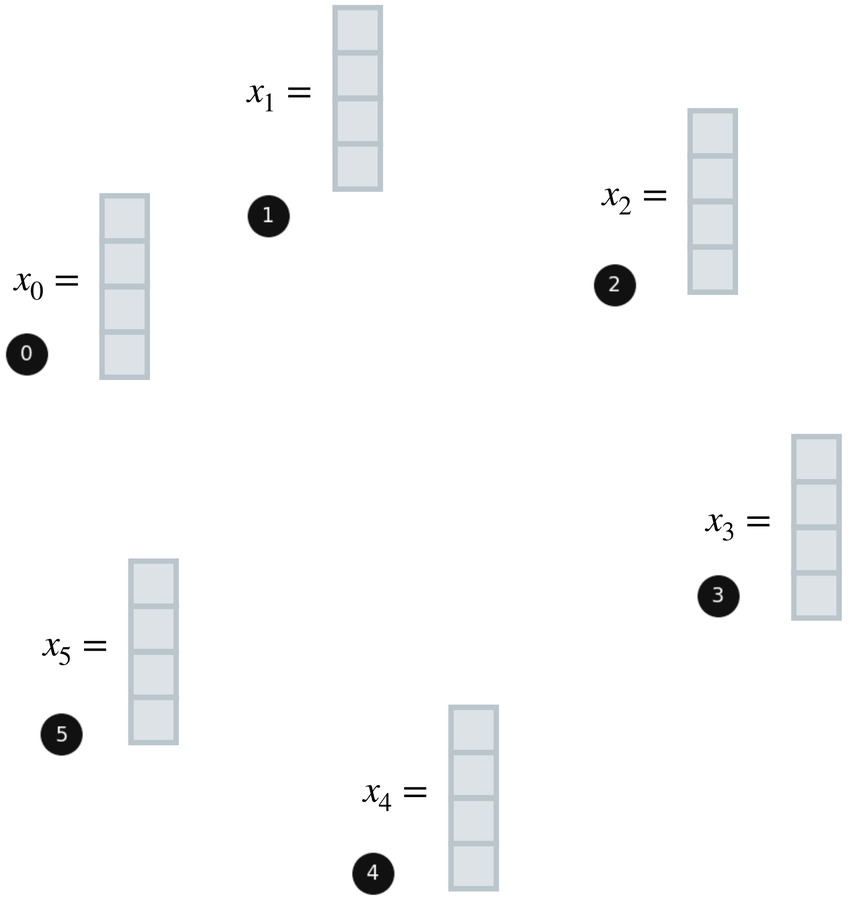
Example:

$$\sigma = (1\,3\,2\,4) \in S_4$$

$$\sigma = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 3 & 4 & 2 & 1 \end{pmatrix}$$

$$P_\sigma = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix}, \; X = \begin{pmatrix} x_1^\top \\ x_2^\top \\ x_3^\top \\ x_4^\top \end{pmatrix}, \quad P_\sigma X = \begin{pmatrix} x_3^\top \\ x_4^\top \\ x_2^\top \\ x_1^\top \end{pmatrix}$$

The DeepSet architecture



Goal: define a function $f : \mathbb{R}^{n \times d} \to \mathbb{R}^k$ taking as input

$$X = (x_1, \ldots, x_n)^\top \in \mathbb{R}^{n \times d}$$

which is *permutation* invariant, that is, which for all permutation matrices P

$$f(PX) = f(X)$$

Theorem

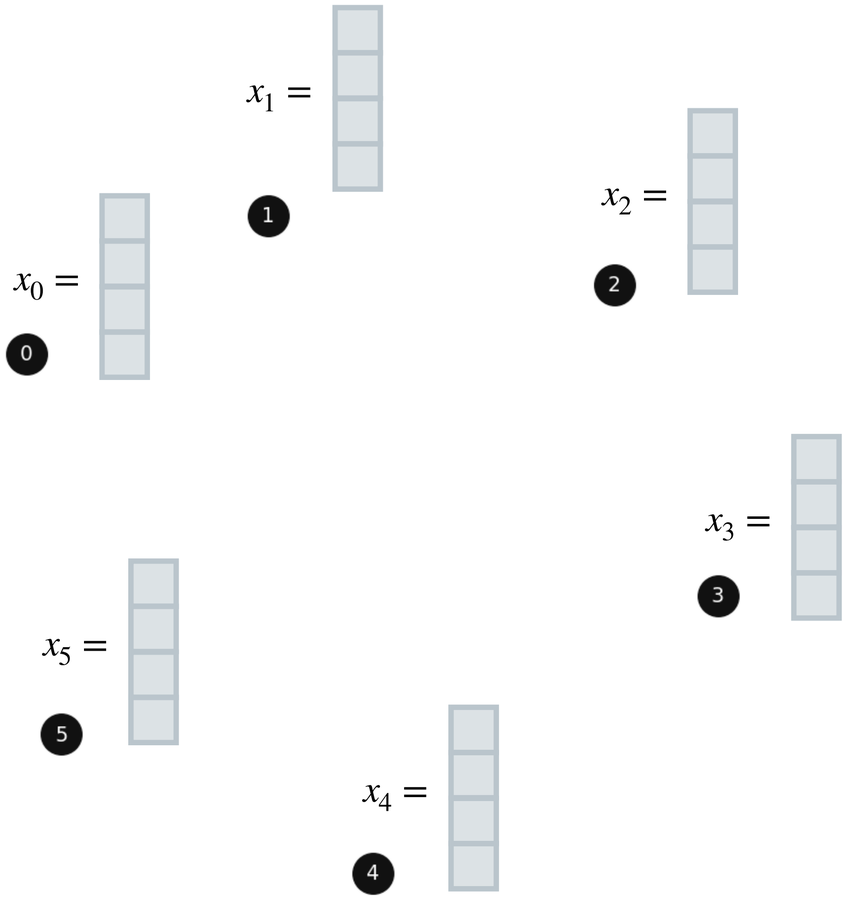
Any such function can be written as

$$f(X) = \varphi\left(\sum_{i \in S} \phi(\mathbf{x}_i)\right)$$

for suitable $\phi : \mathbb{R}^d \to \mathbb{R}^m, \quad \varphi : \mathbb{R}^m \to \mathbb{R}^k$

[1]Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R. R., and Smola, A. (2017). Deep Sets. *Advances in Neural Information Processing Systems*, *30*.

The DeepSet architecture



DeepSets are the architecture built in this manner, to learn from sets

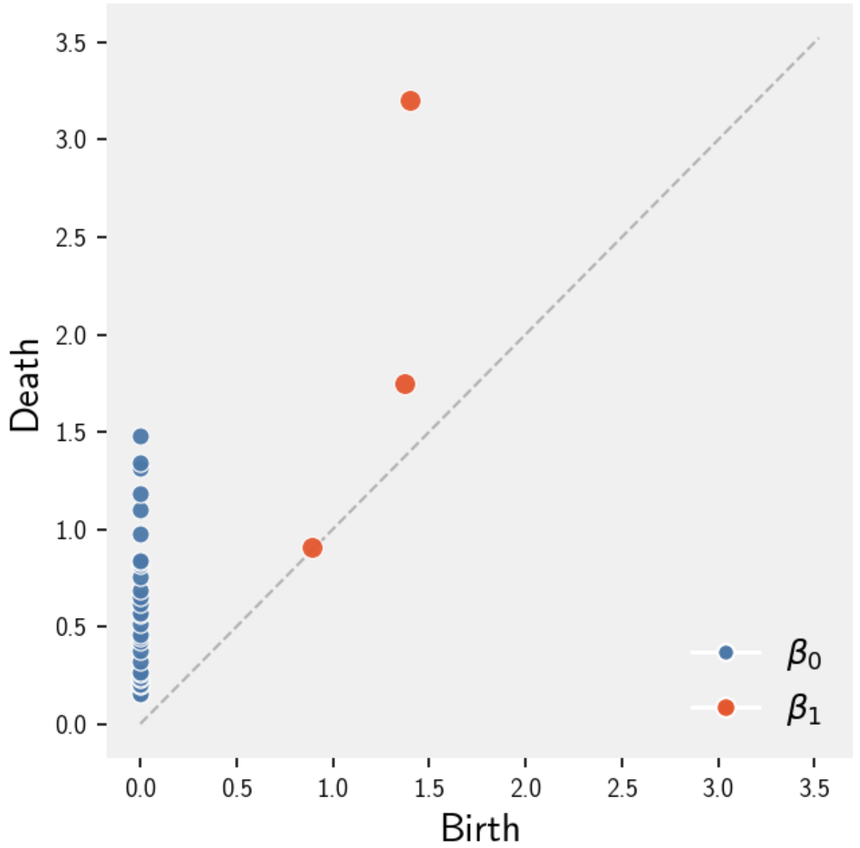
$$f(X) = \varphi_{\theta_2}\left(\sum_{i \in S} \phi_{\theta_1}(\mathbf{x}_i)\right)$$

$$\phi_{\theta_1} : \mathbb{R}^d \to \mathbb{R}^m, \quad \varphi_{\theta_2} : \mathbb{R}^m \to \mathbb{R}^k \qquad \text{MLPs}$$

Substituting the sum by any other permutation invariant operator like taking the maximum, the kth maximum, or an average, you still obtain a permutation invariant function

[1]Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R. R., and Smola, A. (2017). Deep Sets. *Advances in Neural Information Processing Systems*, *30*.

PersLay



Extend the DeepSet architecture to accommodate persistence diagrams

$$\mathrm{PersLay}(\mathrm{PD}) := \bigoplus_{p \in \mathrm{PD}} w_{\theta_1}(p) \cdot \varphi_{\theta_2}(p)$$

With $\bigoplus$ a permutation invariant operator, $w_{\theta_1} : \mathbb{R}^2 \to \mathbb{R}$ a weight function for the points in the diagram and $\varphi_{\theta_2} : \mathbb{R}^2 \to \mathbb{R}^q$ a *point transformation* (a vectorization)

This general formulation allows to recover several popular vectorizations

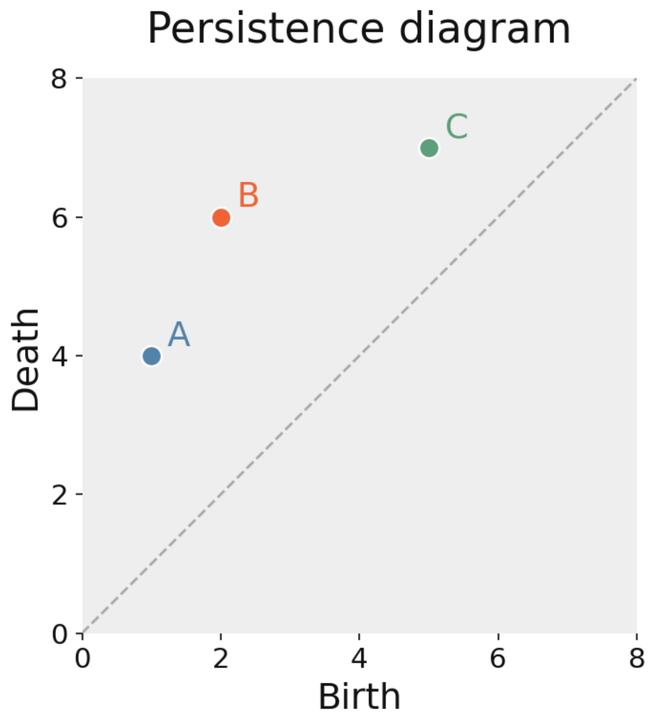
Carrière, M., Chazal, F., Ike, Y., Lacombe, T., Royer, M., and Umeda, Y. (2020). PersLay: A Neural Network Layer for Persistence Diagrams and New Graph Topological Signatures. *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, 2020.

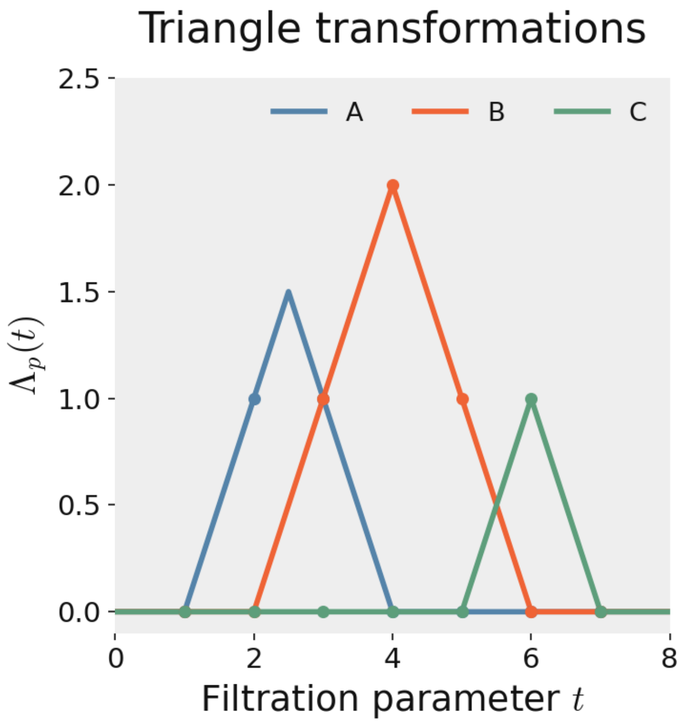
Point transformations

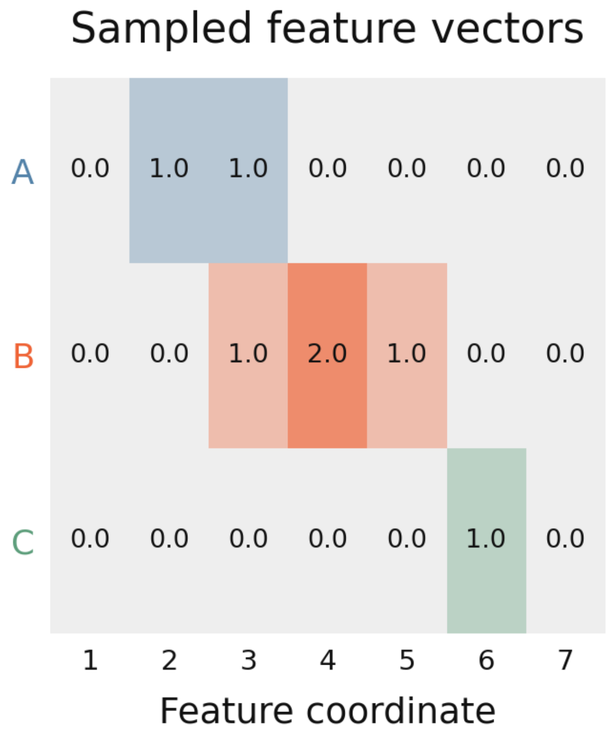
Triangle point transformation:

$$\phi_\Lambda : \mathbb{R}^2 \to \mathbb{R}^q, \qquad p = (x, y) \mapsto \big(\Lambda_p(t_1), \ldots, \Lambda_p(t_q)\big)$$

where $\Lambda_p : \mathbb{R} \to \mathbb{R},\ t \mapsto \max\{0,\ y - |t - x|\}$, for $q \in \mathbb{N},\ t_1, \ldots, t_q \in \mathbb{R}$ (learnable parameters)





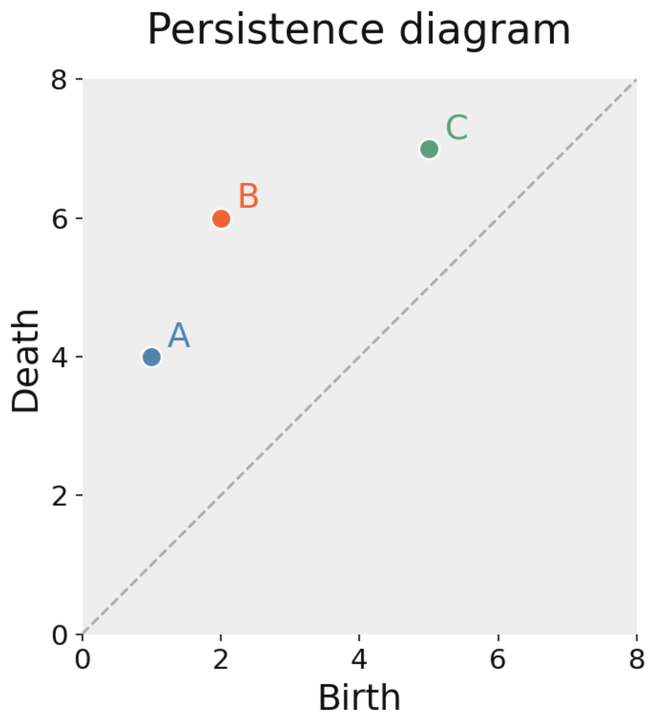


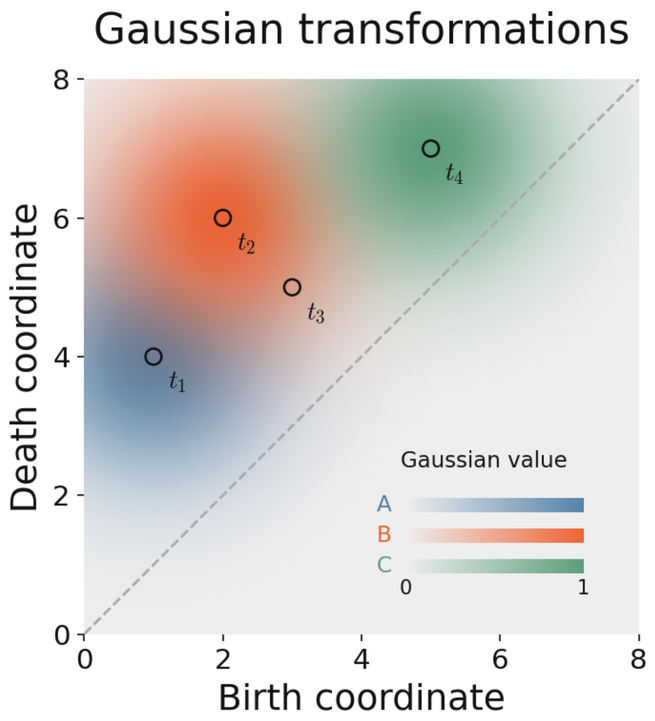
Point transformations

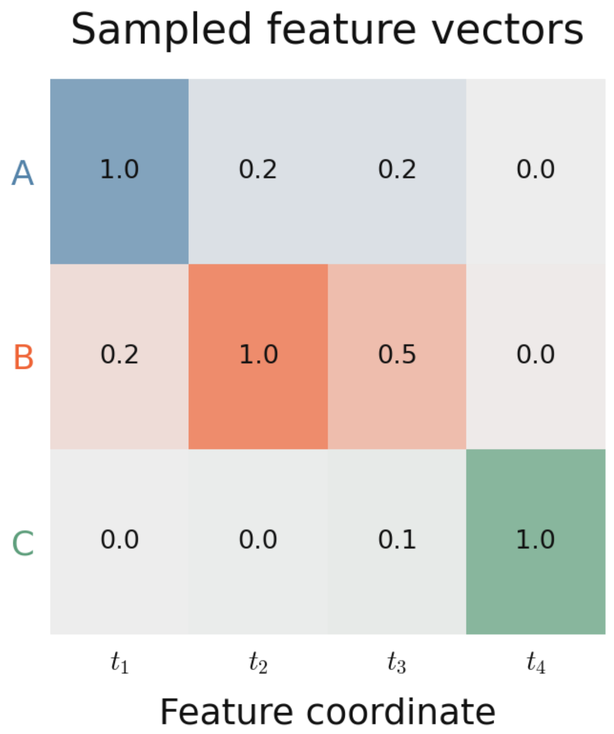
Gaussian point transformation:

$$\phi_\Gamma : \mathbb{R}^2 \to \mathbb{R}^q, \qquad p = (x, y) \mapsto \big(\Gamma_p(t_1), \ldots, \Gamma_p(t_q)\big)$$

where $\Gamma_p : \mathbb{R}^2 \to \mathbb{R}, \quad t \mapsto \exp\left(\frac{-\|p-t\|^2}{2\sigma^2}\right)$, for $\sigma > 0$ (learnable parameter) and $q \in \mathbb{N}$, $t_1, \ldots, t_q \in \mathbb{R}^2$ (fixed)





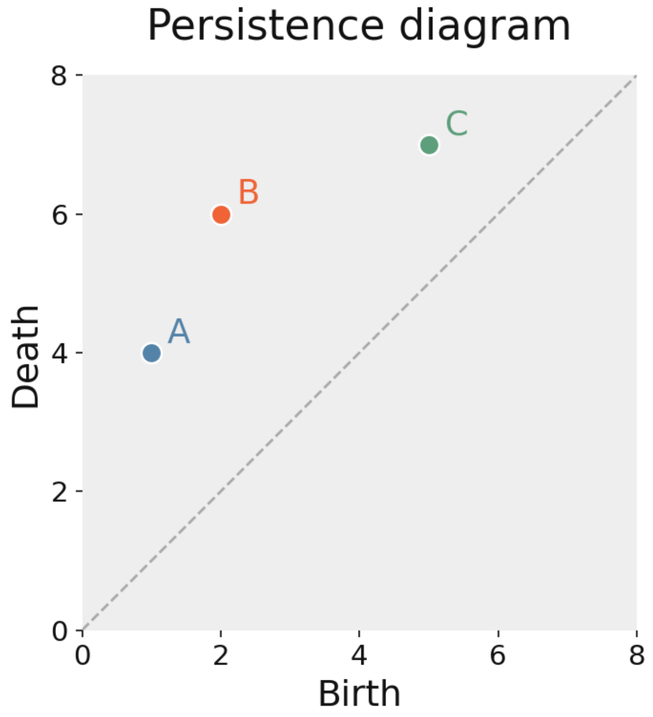


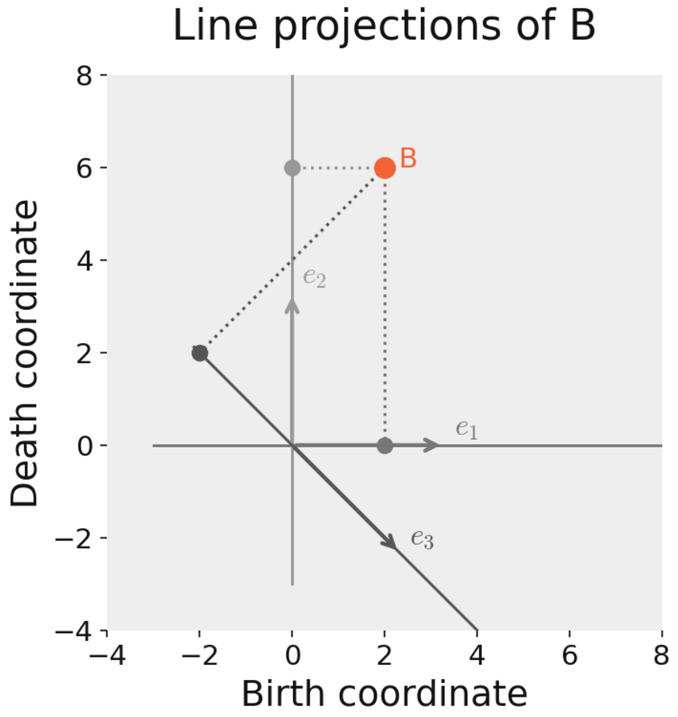
Point transformations

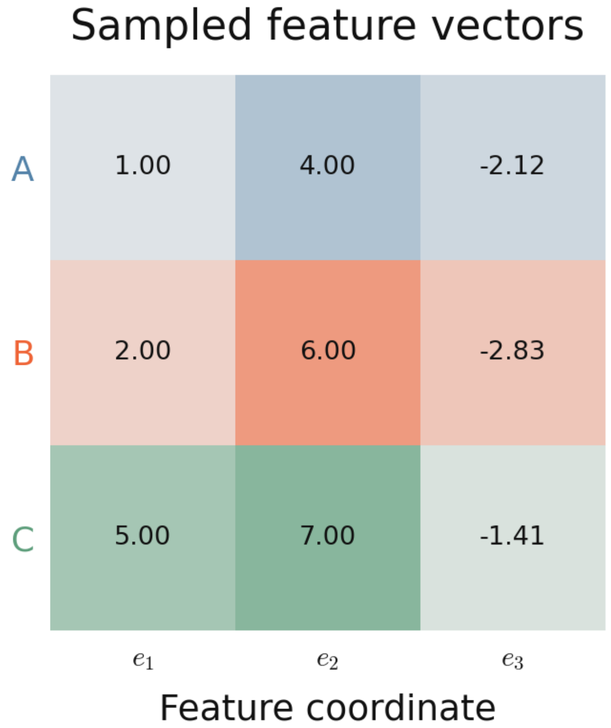
Line point transformation:

$$\phi_L : \mathbb{R}^2 \to \mathbb{R}^q, \qquad p = (x, y) \mapsto \left(L_{\Delta_1}(p), \ldots, L_{\Delta_q}(p)\right)$$

where $L_\Delta : \mathbb{R}^2 \to \mathbb{R}, \quad p \mapsto \langle p, e_\Delta \rangle + b_\Delta,$ for the line Δ with direction vector $e_\Delta \in \mathbb{R}^2$ and bias $b_\Delta \in \mathbb{R}$, $q \in \mathbb{N}$ and $\Delta_1, \ldots, \Delta_q$ are q lines on the plane



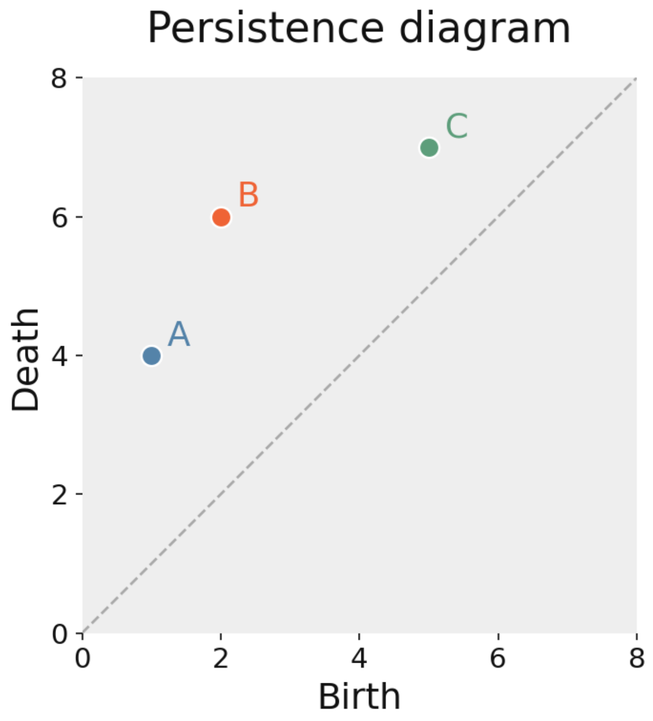


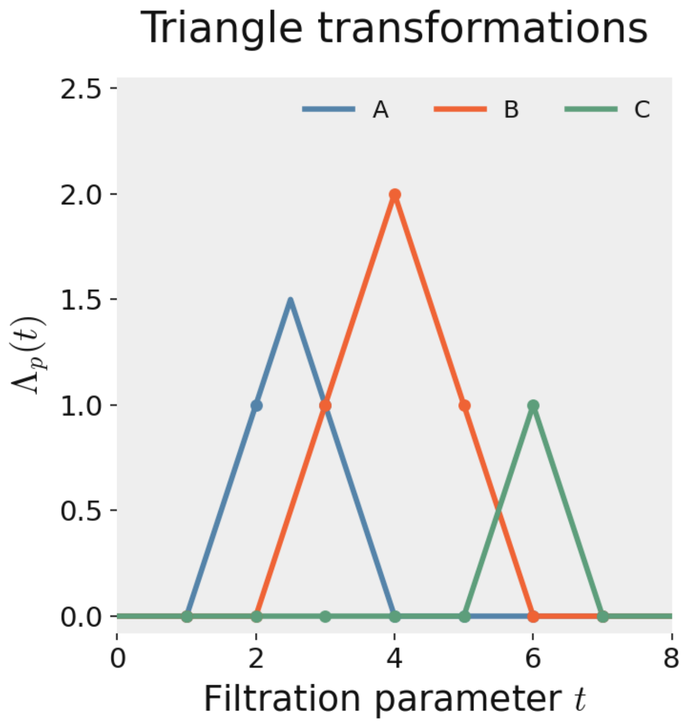


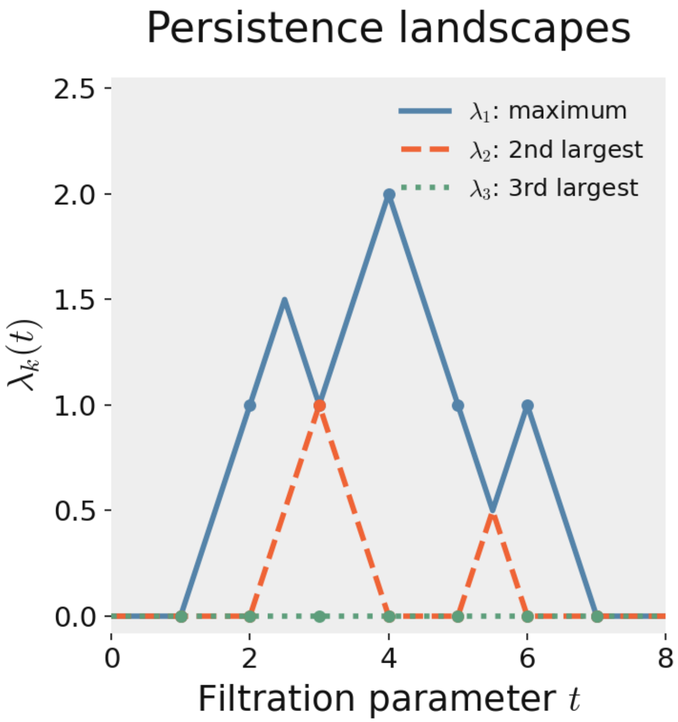
Recovering persistence landscapes

$$\phi = \phi_\Lambda \text{ (triangle point transformation)}, \quad w(p) = 1, \forall p, \quad \bigoplus = k\text{-th largest value per coordinate}$$

$$\left(\operatorname*{kmax}_{p \in \mathrm{PD}} \Lambda_p(t_1), \ldots, \operatorname*{kmax}_{p \in \mathrm{PD}} \Lambda_p(t_q) \right)\}$$







Application to graph classification

Input data: a graph $G = (V, E)$ seen as a discrete metric space (V, d) with V set of vertices in the graph and d a graph distance

Def. Shortest path distance

Let $G = (V, E)$ be an undirected graph, and $u, v \in V$ two vertices. A *path* between them is a collection of edges

$$\tau = \{e_i = (u_i, u_{i+1}) : 0 \leq i \leq n-1\}$$

with $u = u_0$ and $v = u_n$

The *length* of such a path is $n = \text{len}(\tau)$.

The *shortest path distance* between the vertices is defined as the minimal length among paths between them:

$$d_G : V \times V \to \mathbb{R}_{\geq 0} \qquad d_G(u, v) = \min_{\tau \text{ path}} \text{len}(\tau)$$

Application to graph classification

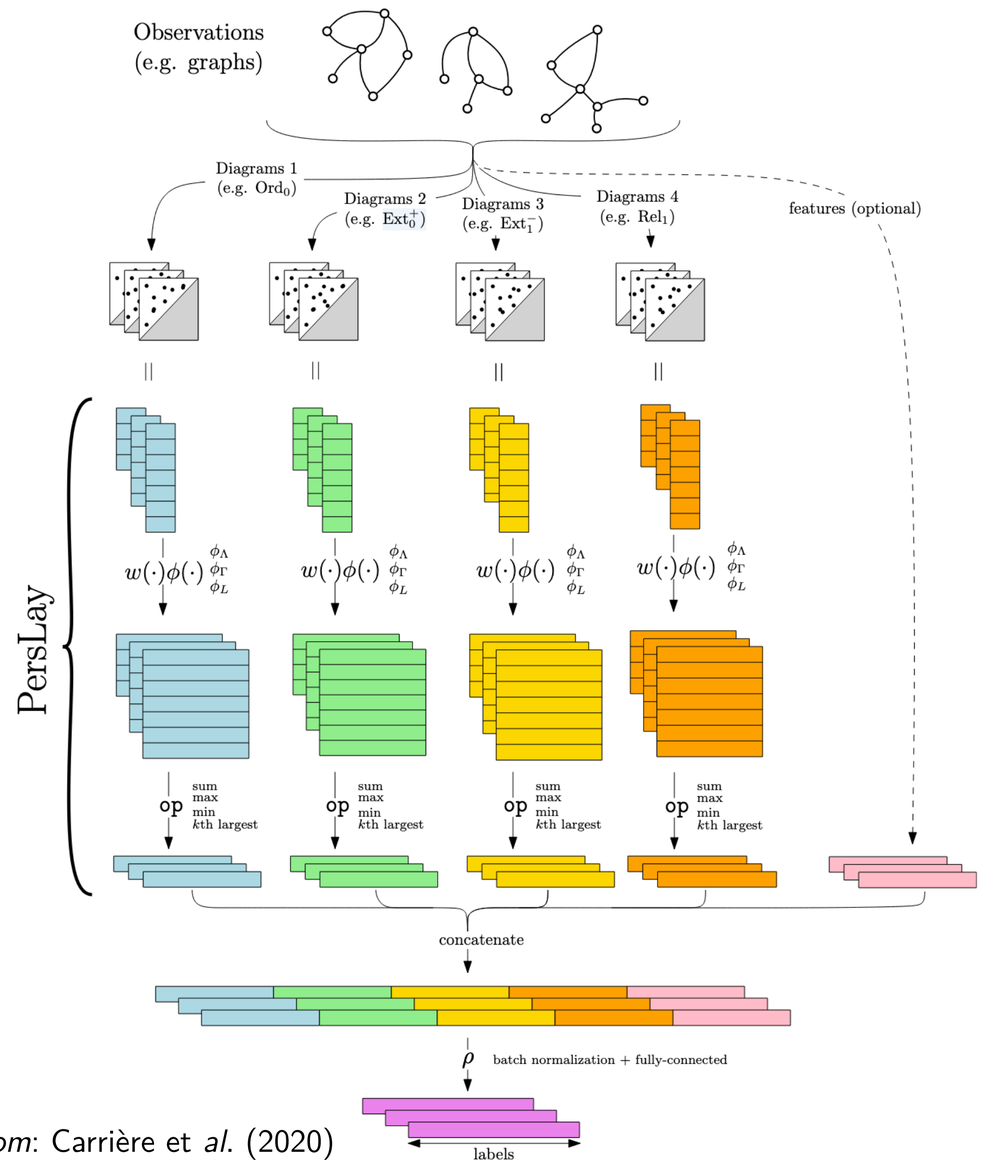


Figure from: Carrière et *al.* (2020)

Input data: graphs, seen as discrete metric spaces with a distance defined on the vertices according to the graph structure

Compute **persistence diagrams** from *extended persistence*

Apply the `PersLay` transformation for three point transformations and learnable weights

Concatenate all resulting vectors + normalize

Apply an **MLP** to classify

Datasets studied for the graph application

Dataset	Nb graphs	Nb classes	Av. nodes	Av. Edges	Av. β_0	Av. β_1
REDDIT5K	5,000	5	508.5	594.9	3.71	90.1
REDDIT12K	12,000	11	391.4	456.9	2.8	68.29
COLLAB	5,000	3	74.5	2457.5	1.0	2383.7
IMDB-B	1,000	2	19.77	96.53	1.0	77.76
IMDB-M	1,500	3	13.00	65.94	1.0	53.93
COX2	467	2	41.22	43.45	1.0	3.22
DHFR	756	2	42.43	44.54	1.0	3.12
MUTAG	188	2	17.93	19.79	1.0	2.86
PROTEINS	1,113	2	39.06	72.82	1.08	34.84
NCI1	4,110	2	29.87	32.30	1.19	3.62
NCI109	4,127	2	29.68	32.13	1.20	3.64

Social graphs

Molecular datasets

Table from: Carrière et *al.* (2020)

Results

Dataset	SV[1]	RetGK* [2]	FGSD [3]	GCNN [4]	GIN [5]	PersLay	
						Mean	Max
REDDIT5K	—	56.1	47.8	52.9	57.0	55.6	56.5
REDDIT12K	—	48.7	—	46.6	—	47.7	49.1
COLLAB	—	81.0	80.0	79.6	80.1	76.4	78.0
IMDB-B	72.9	71.9	73.6	73.1	74.3	71.2	72.6
IMDB-M	50.3	47.7	52.4	50.3	52.1	48.8	52.2
COX2*	78.4	80.1	—	—	—	80.9	81.6
DHFR*	78.4	81.5	—	—	—	80.3	80.9
MUTAG*	88.3	90.3	92.1	86.7	89.0	89.8	91.5
PROTEINS*	72.6	75.8	73.4	76.3	75.9	74.8	75.9
NCI1*	71.6	84.5	79.8	78.4	82.7	73.5	74.0
NCI109*	70.5	—	78.8	—	—	69.5	70.1

Table from: Carrière et *al.* (2020)

- SV: kernel method on persistence diagrams for point clouds obtained from the graphs
- RetBK: graph kernel method using graph attributes
- FGSD: graph kernel method that doesn't use graph attributes
- GCNN, GIN: graph neural networks

Limitations

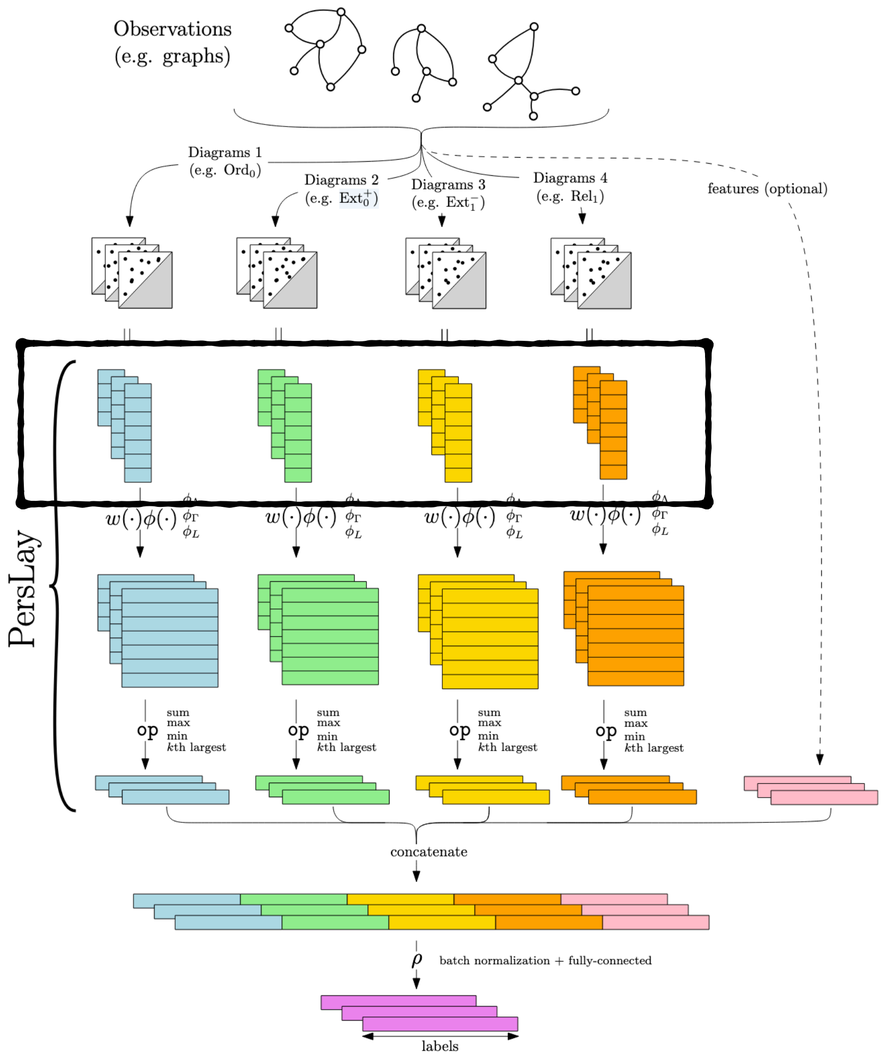


Figure from: Carrière et *al.* (2020)

Not learnable: the persistence computation is fixed, can we make it learnable?

Can we differentiate wrt any parameters here?

Learnable part: where the parameters that we are optimizing live

Topological losses: learning the filtration

Lecture 9 — Topological losses

Defining a loss on persistence diagrams

Persistence diagram: $\mathrm{PD}_k = \{(b_i, d_i) : i \in \mathcal{I}_k\}$

Order points by decreasing lifetime $d_i - b_i \geq d_j - b_j \iff i < j$

Goal: define a *differentiable* loss of these diagrams

$$\mathcal{L}(p, q, i_0; \mathrm{PD}_k) = \sum_{i=i_0}^{|\mathcal{I}_k|} |d_i - b_i|^p \left(\frac{d_i + b_i}{2}\right)^q$$

Leave the first i_0 most persistent features out

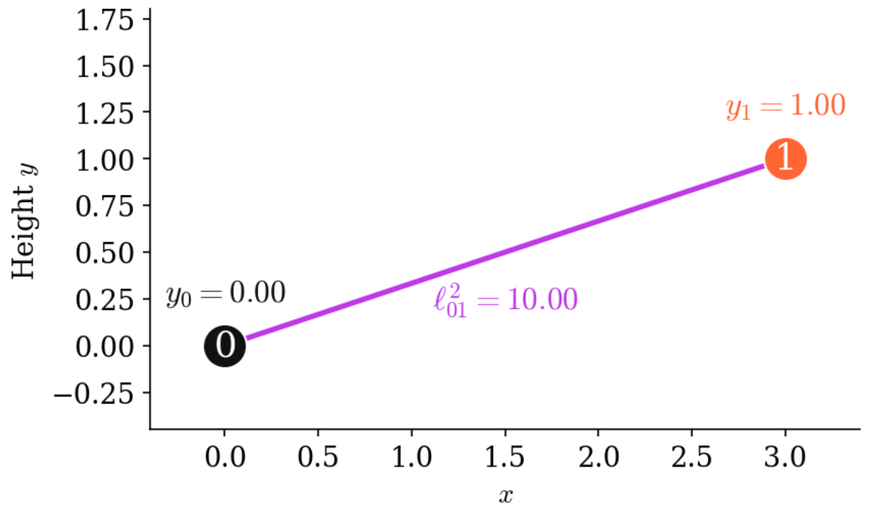
Increasing p penalizes more the most persistent features

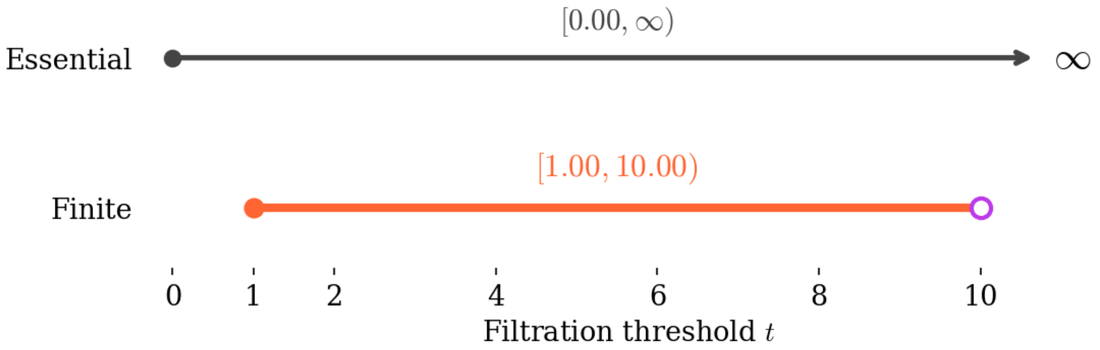
Weight that gives more importance to bars appearing later in the filtration

Goal: we want to compute the derivative of the loss with respect to the birth and death values

Gabrielsson, R. B., Nelson, B. J., Dwaraknath, A., and Skraba, P. (2020). A Topology Layer for Machine Learning. *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, 1553–1563.

Computing the gradient of the loss: example





Consider the following filtration over the point cloud:

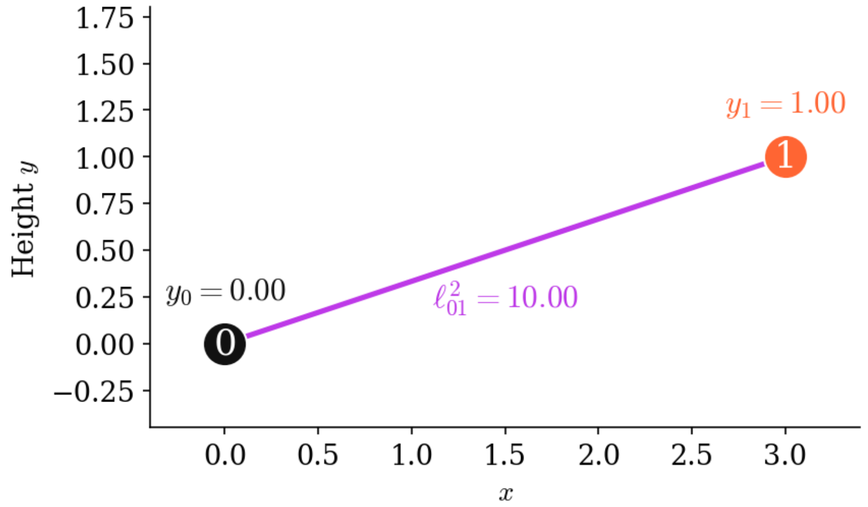
$$P = (p_0, p_1) \subset \mathbb{R}^2$$

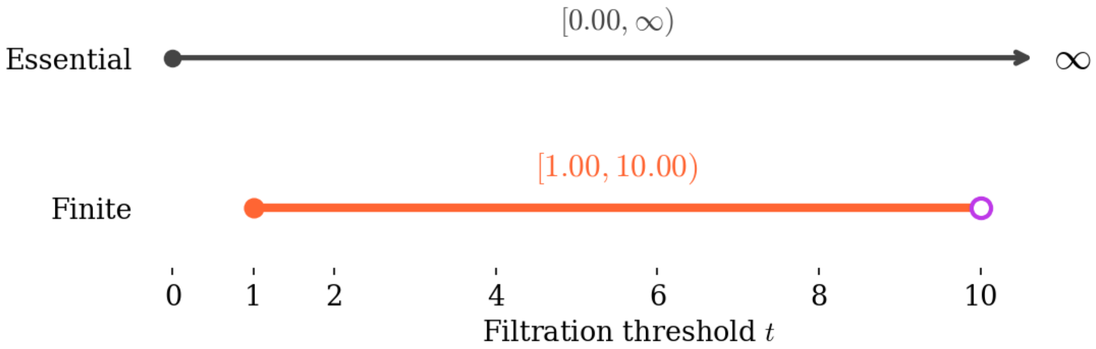
- Points enter the filtration according to their height
- The edge enters the filtration according to the distance between the points

$$\mathcal{L}(2, 0, 2; \mathrm{PD}_0) = (d - b)^2$$

Try it: can you express the loss in terms of differentiable variables?

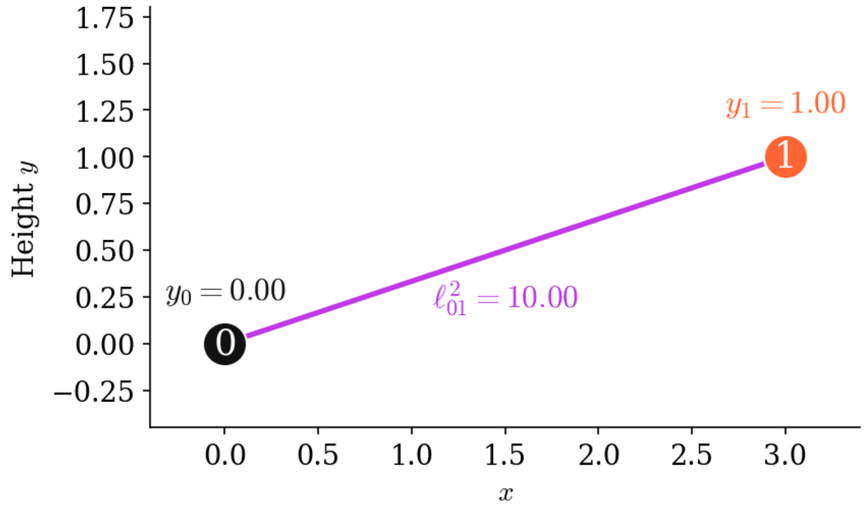
Computing the gradient of the loss: example

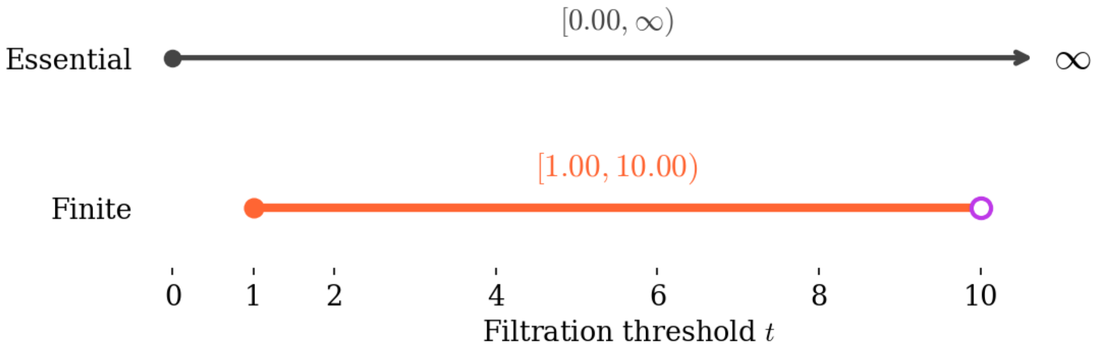




Try it: can you compute the derivatives of the loss wrt d and b?

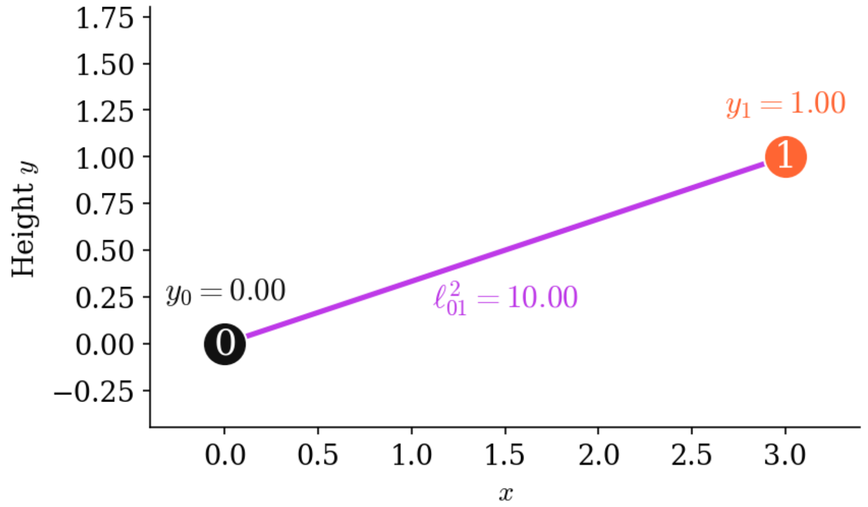
Computing the gradient of the loss: example

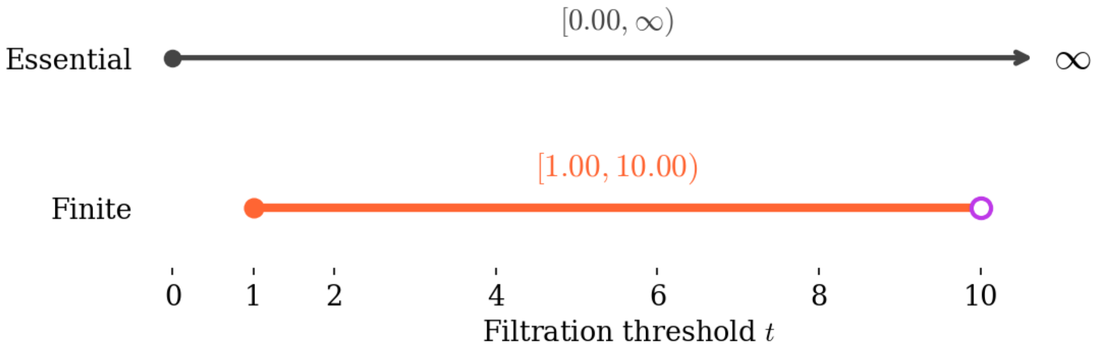




Try it: can you use the previous computation to obtain the gradient of the loss wrt (x_0, x_1, y_0, y_1)?

Computing the gradient of the loss: example

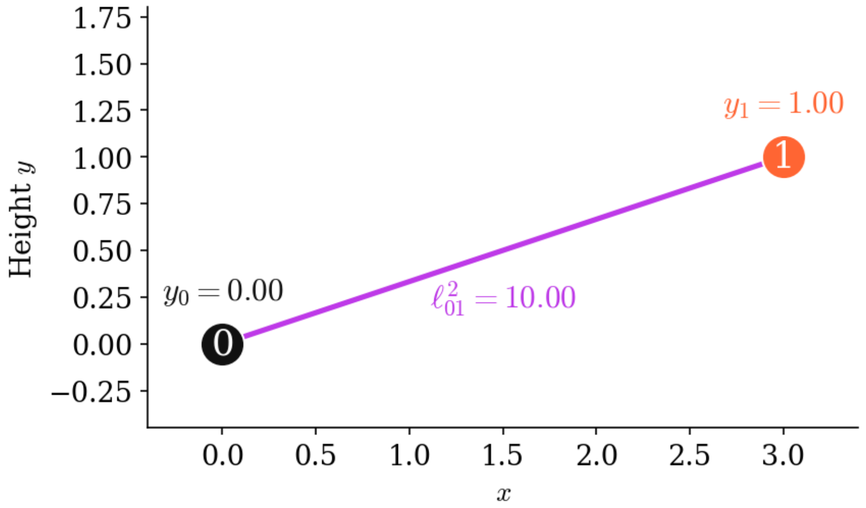


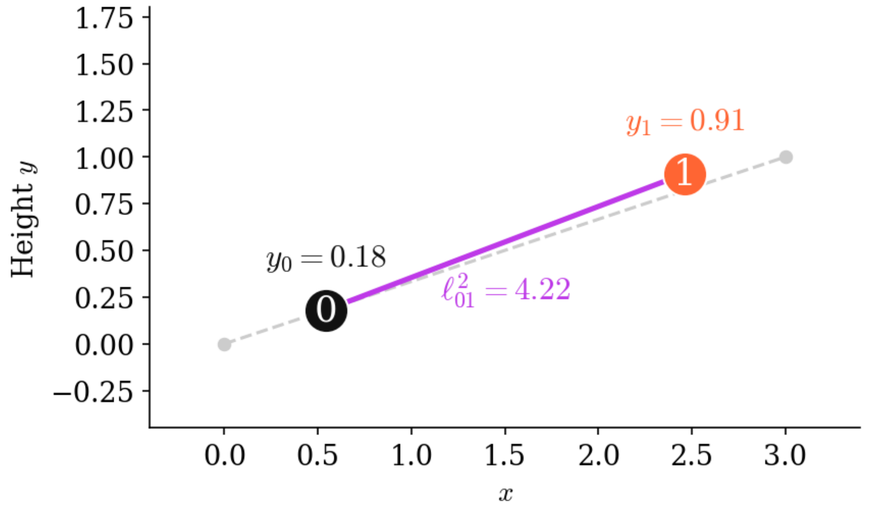


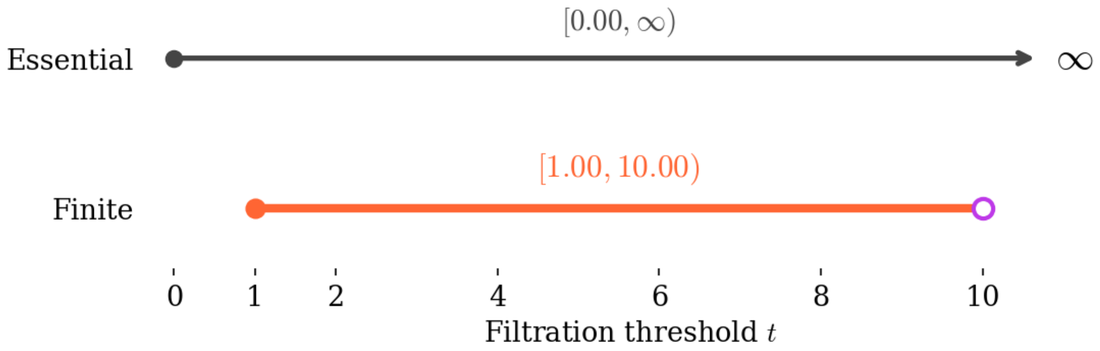
Try it: can you use evaluate that gradient on our given configuration?

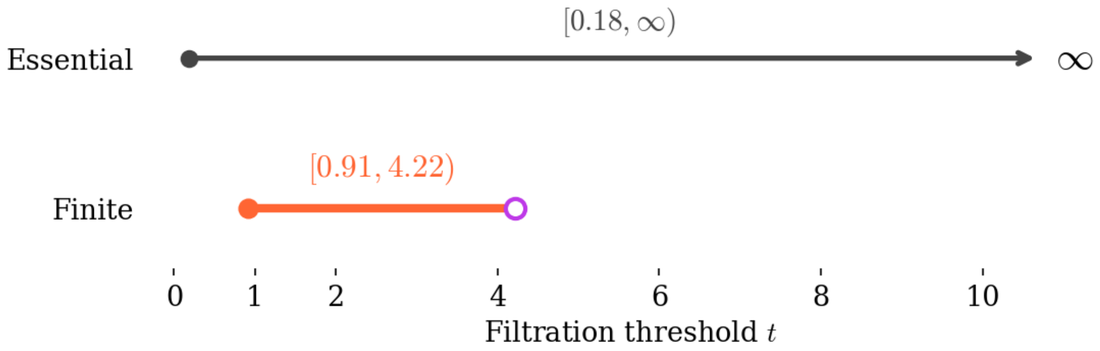
Computing the gradient of the loss: example

$$P_{\text{new}} \mapsto P - 0.005\ \nabla_P \mathcal{L}$$









What did we need for this to work?

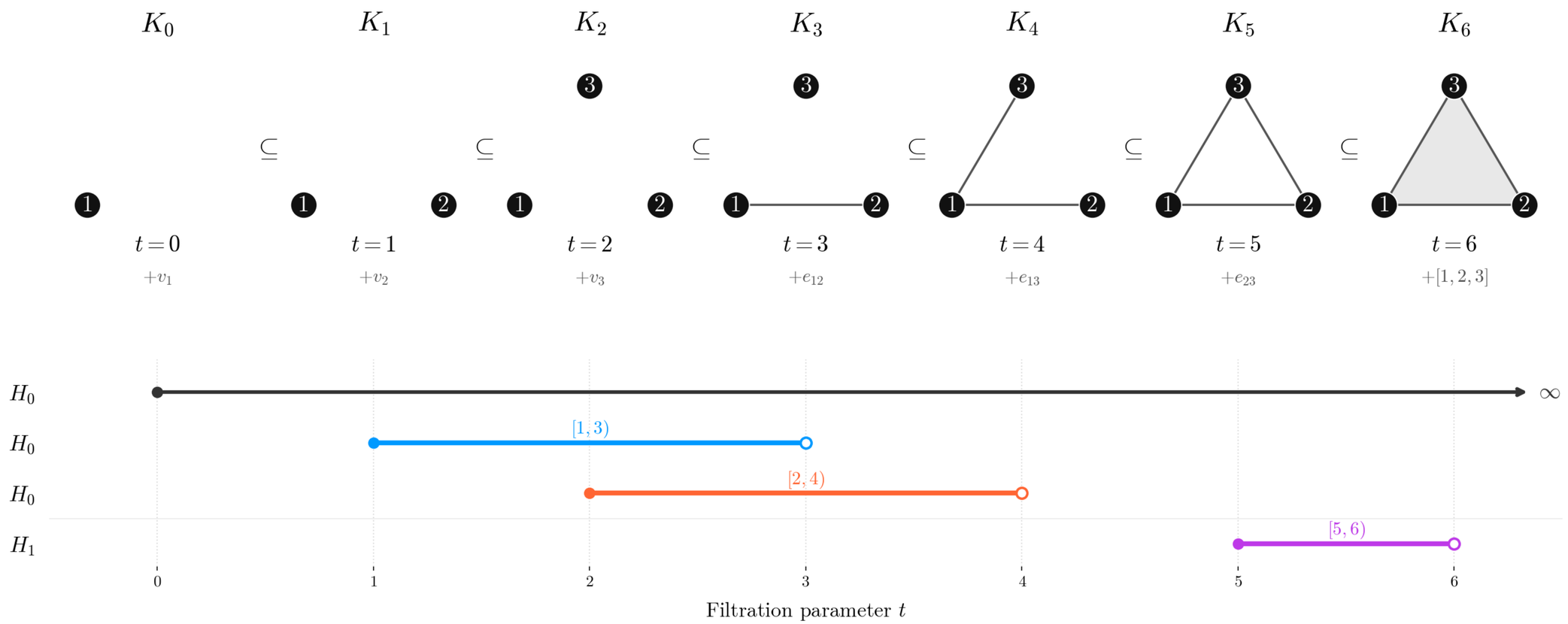
- We needed to match each birth and death time in a bar to a differentiable parameter given by the positions of the points
- In other words, we have matched each bar to the simplex that generated it (critical simplex)
- And then we have identified that simplex to its filtration value, obtained through differentiable position parameters

In the background, there is a big assumption that we can match uniquely bars to critical simplices, which is not true in general

Simplex-wise filtrations

Def. Simplex-wise filtration

A filtration $\{X_t : t \in \mathbb{R}\}$ such that at each parameter value $t \in \mathbb{R}$ at most one simplex enters the filtration



Can we extend to general filtrations?

Critical simplices: simplices that created and destroyed the topological feature represented by the bar

If the filtration is simplex-wise, we have a *unique map:*

$$\pi_k : \{(b_i, d_i) : i \in \mathcal{I}_k\} \to (\sigma, \tau)$$

In the background, there is a big assumption that we have a simplex-wise filtration which is not true in general

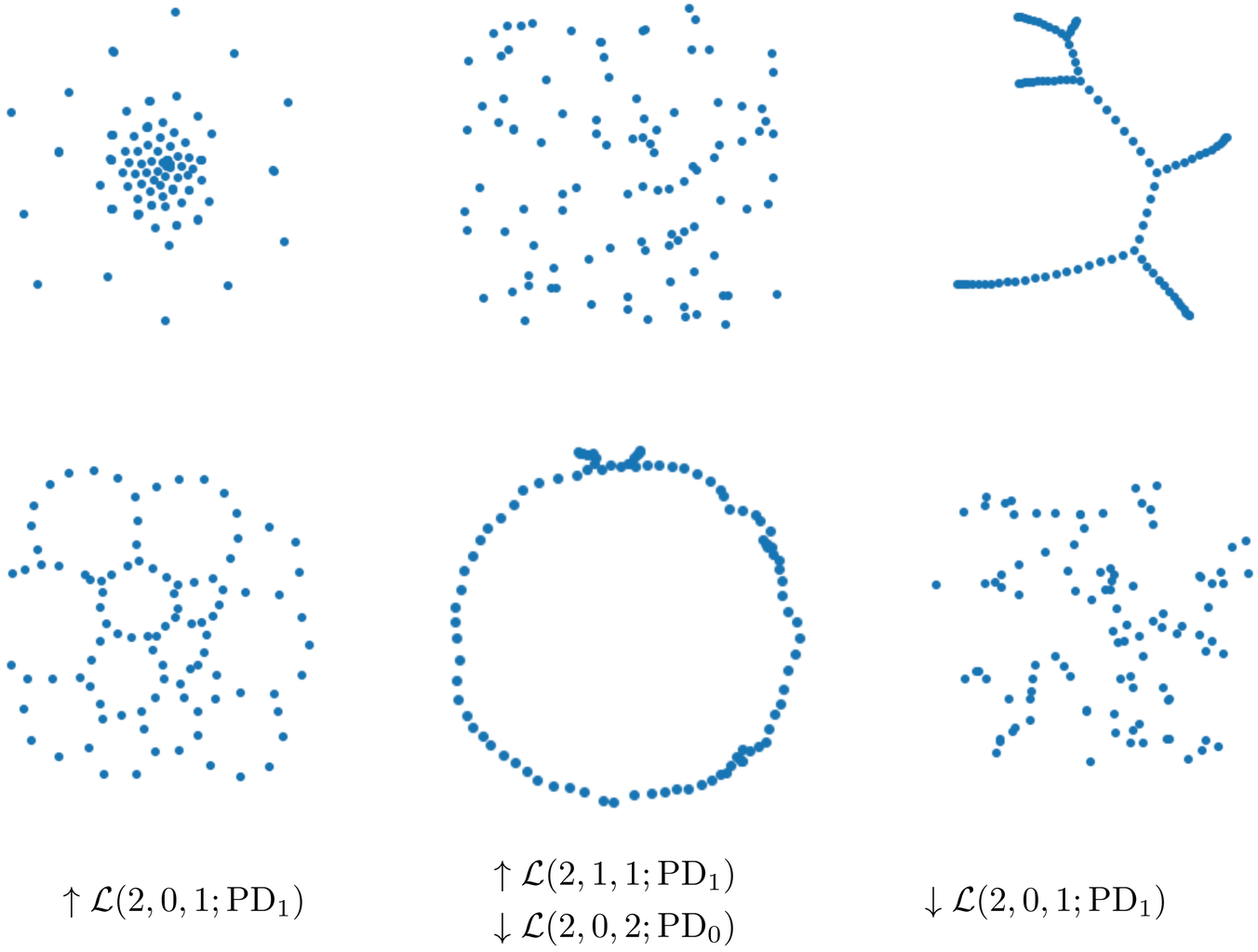
Note: you can extend this to work with general filtrations, by making them simplex-wise choosing a total order of the simplices

In that case, the derivative should be considered as a *subgradient*, and choosing a total order selects an element in the subgradient

General point clouds

Figure from: Gabrielsson et *al.* (2020)

$$\mathcal{L}(2, 0, i_0; \mathrm{PD}_k) = \sum_{i=i_0}^{|\mathcal{I}_k|} |d_i - b_i|^2$$



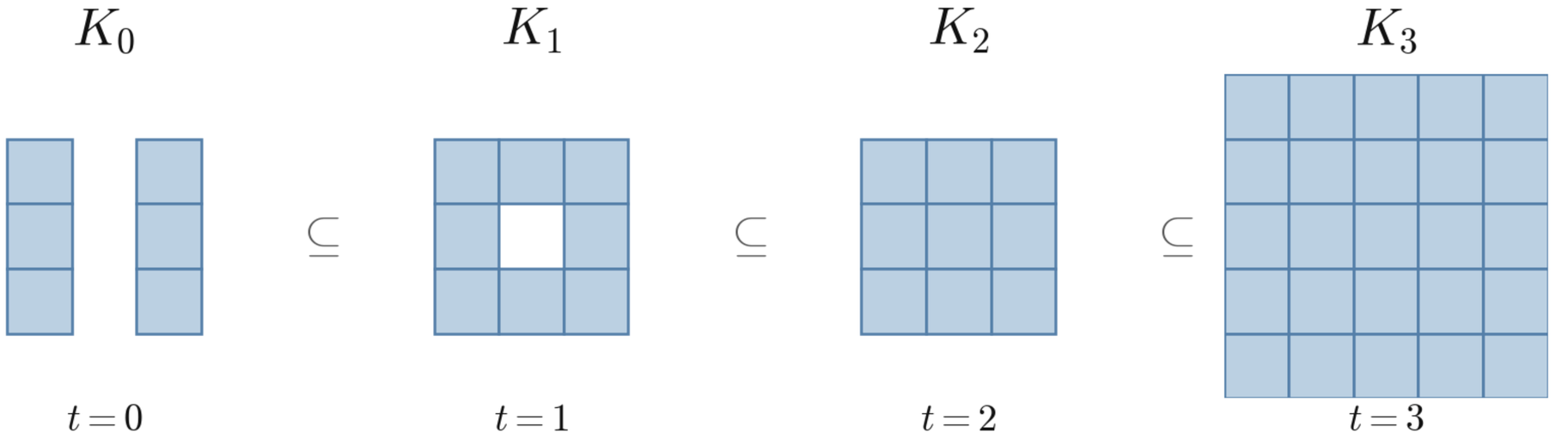
Gabrielsson, R. B., Nelson, B. J., Dwaraknath, A., and Skraba, P. (2020). A Topology Layer for Machine Learning. *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, 1553–1563.

Small detour: PH for images

Cubical complexes filtration $K_t = f^{-1}(-\infty, t]$

Pixel intensities

3	3	3	3	3
3	0	1	0	3
3	0	2	0	3
3	0	1	0	3
3	3	3	3	3





An application to noise reduction

Figure from: Gabrielsson et *al*. (2020)

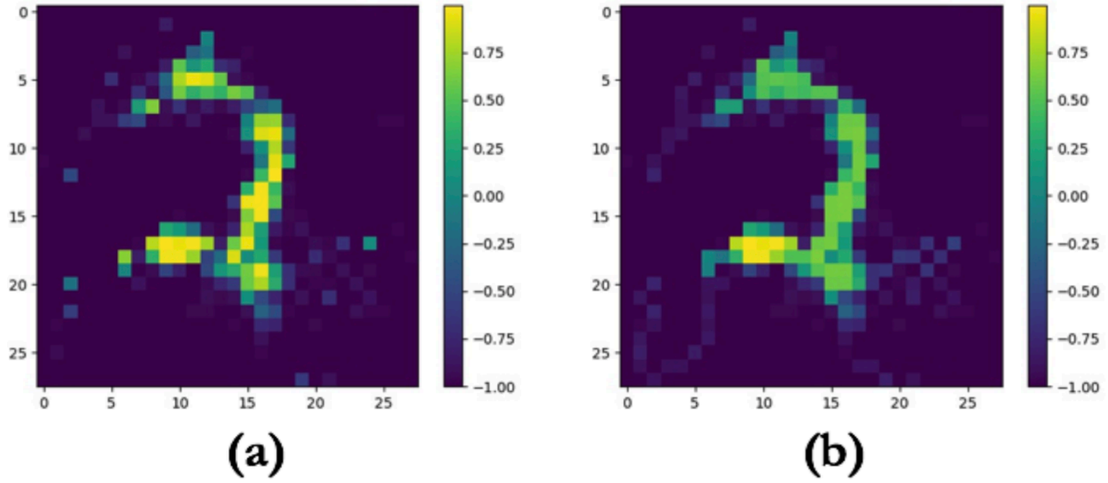


Figure 3: (a) A noisy image of the digit '2'. (b) after promoting a single local maximum using $\mathcal{L}(1, 0, 2; \mathrm{PD}_0)$

Gabrielsson, R. B., Nelson, B. J., Dwaraknath, A., and Skraba, P. (2020). A Topology Layer for Machine Learning. *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, 1553–1563.

An application to image generation

Figure from: Gabrielsson et *al.* (2020)

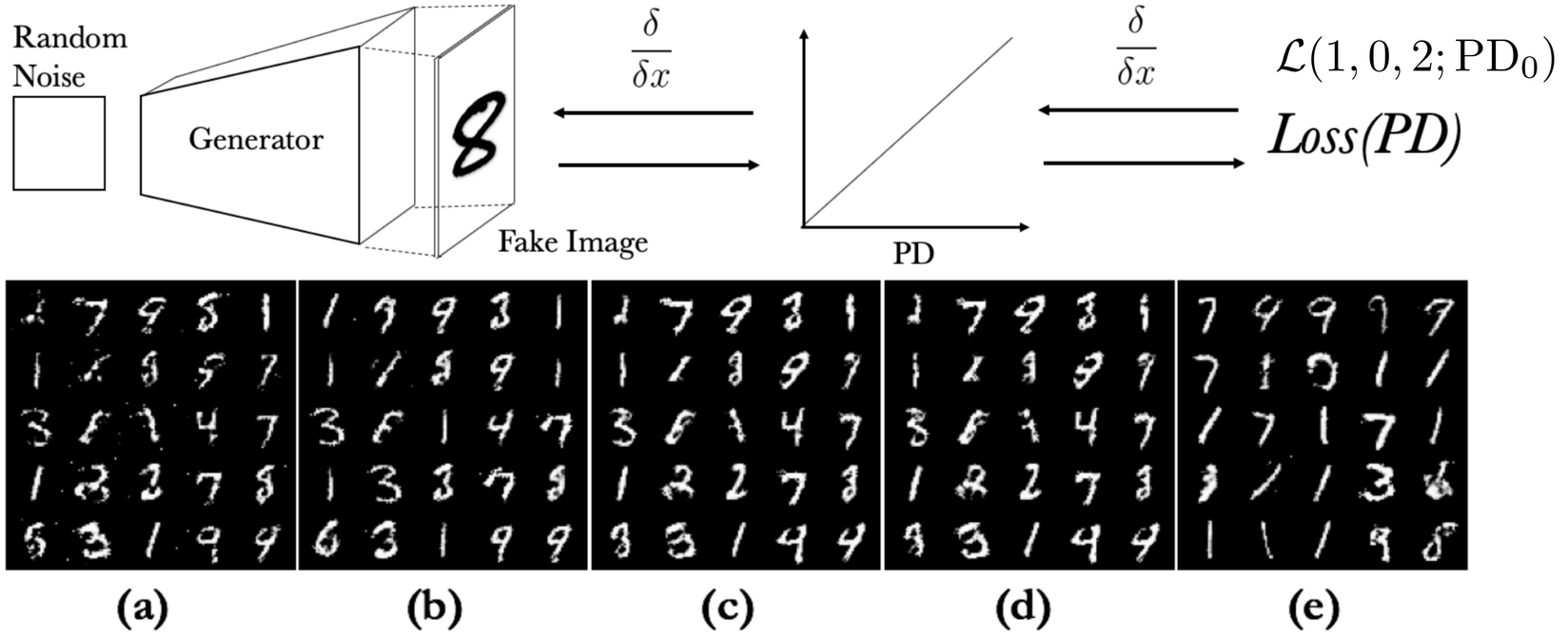
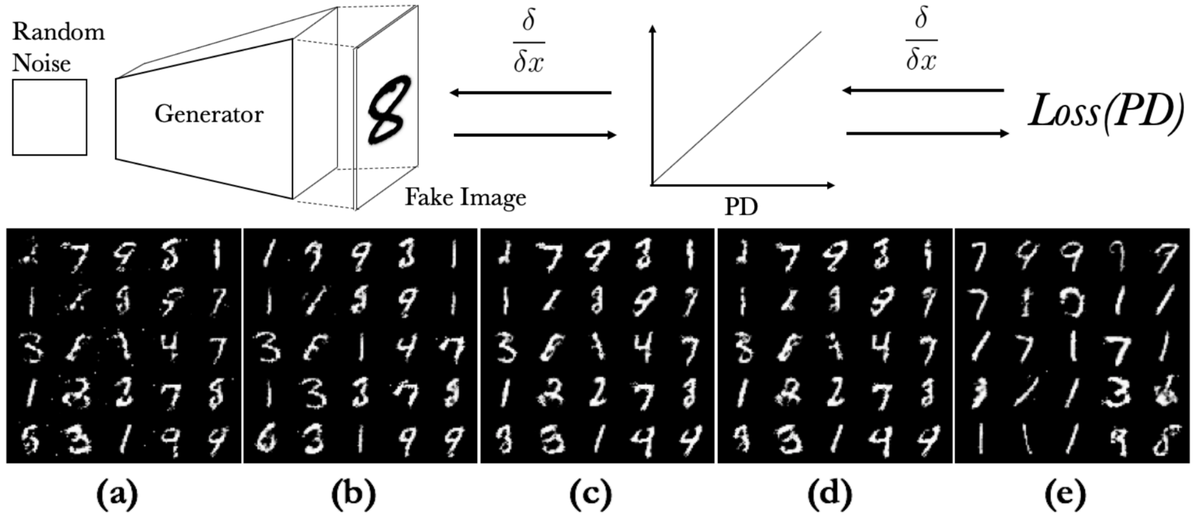


Figure 8: Top: Setup for computing topology loss and backpropogation. (a) Baseline-Generator. (b) Minimize the topology loss with respect to the latent space of Baseline-Generator. (c) Topology-Generator. (d) Train Baseline-Generator with topology-discriminator for 100 batch iterations. (e) Train Baseline-Generator in original GAN-setup for another 60,000 batch iterations.

Gabrielsson, R. B., Nelson, B. J., Dwaraknath, A., and Skraba, P. (2020). A Topology Layer for Machine Learning. *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, 1553–1563.

An application to image generation

Figure from: Gabrielsson et *al*. (2020)



Conclusions:

- Optimizing the latent input helps, but leaves some topological noise. Updating the generator in (c) gives a stronger improvement.
- An explicit topological loss can efficiently improve connectivity, which doesn't naturally happen in the GAN (e)
- The connectivity prior agrees with what is learned from real data: (c) is similar to (d). Good topological prior.

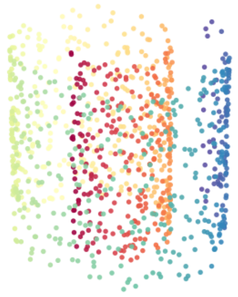
Gabrielsson, R. B., Nelson, B. J., Dwaraknath, A., and Skraba, P. (2020). A Topology Layer for Machine Learning. *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, 1553–1563.

Topological autoencoders

Lecture 9 — Topological losses

Dimensionality reduction

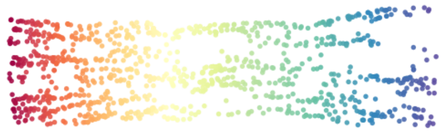
Input: $\mathbb{R}^3$



PCA



Isomap



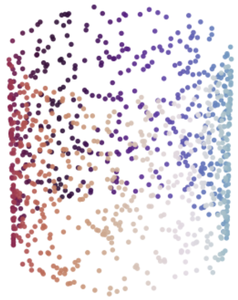
t-SNE



UMAP



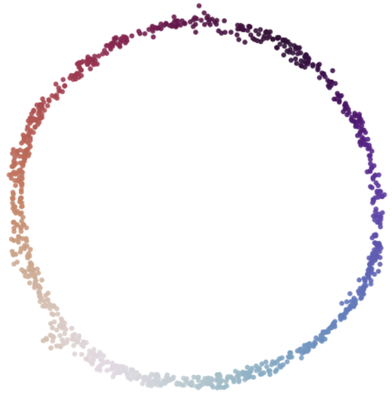
Input: $\mathbb{R}^3$



PCA



Isomap



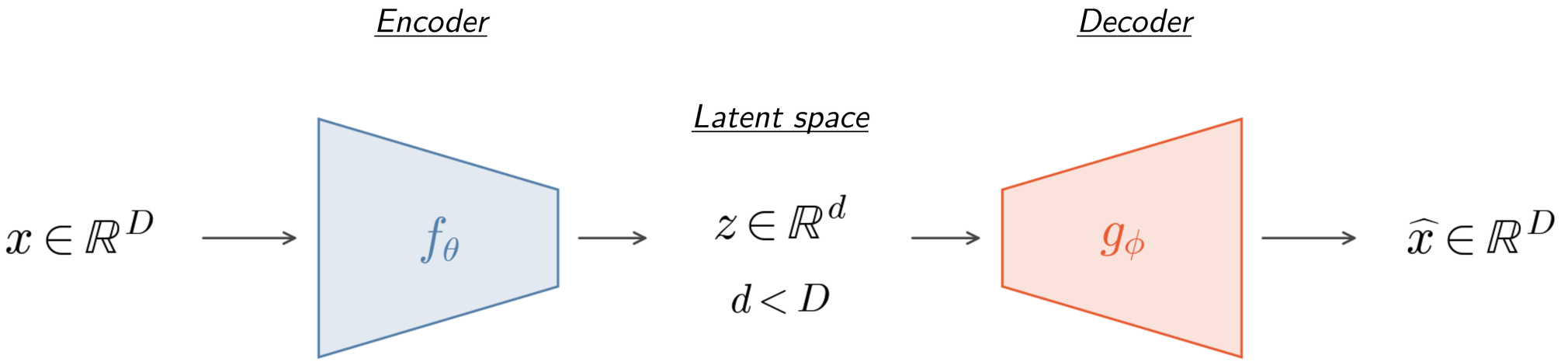
t-SNE



UMAP



Autoencoders

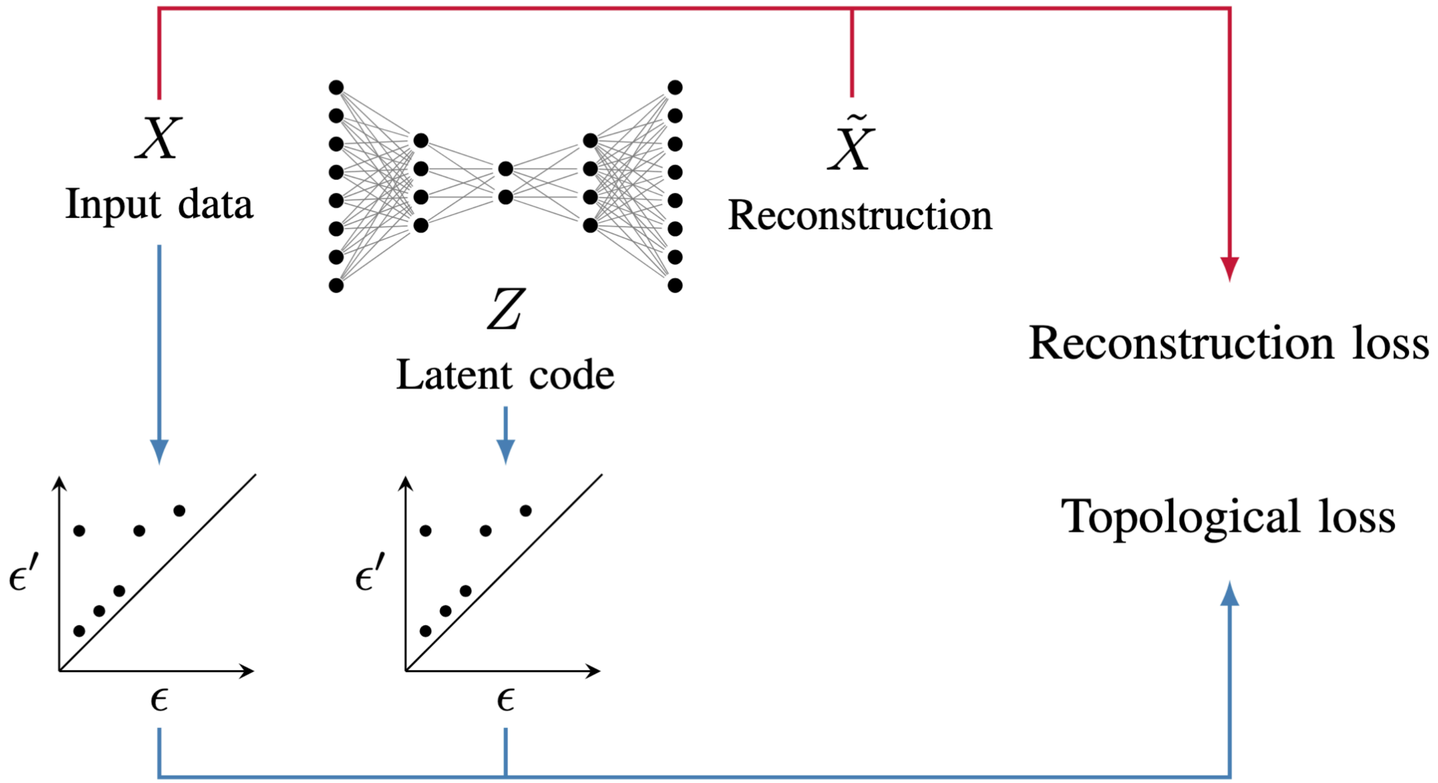


$$\mathcal{L}_{\text{rec}}(\theta, \phi) = \frac{1}{n} \sum_{i=1}^{n} \| x_i - g_\phi(f_\theta(x_i)) \|^2$$

Reconstruction loss

A topological loss for auto encoders

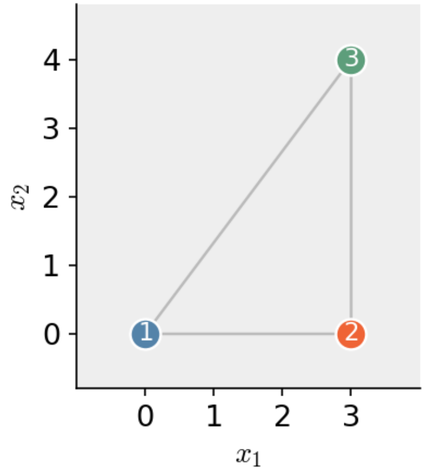
Figure from: Moor et *al.* (2020)



Moor, M., Horn, M., Rieck, B., and Borgwardt, K. (2020). Topological autoencoders. In *International conference on machine learning* (pp. 7045-7054). PMLR.

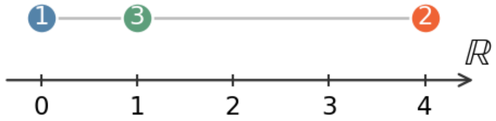
Computing diagrams and matchings

X



f_θ

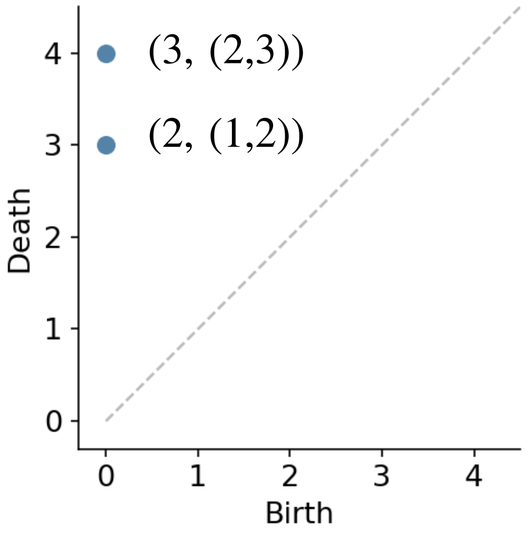
Z

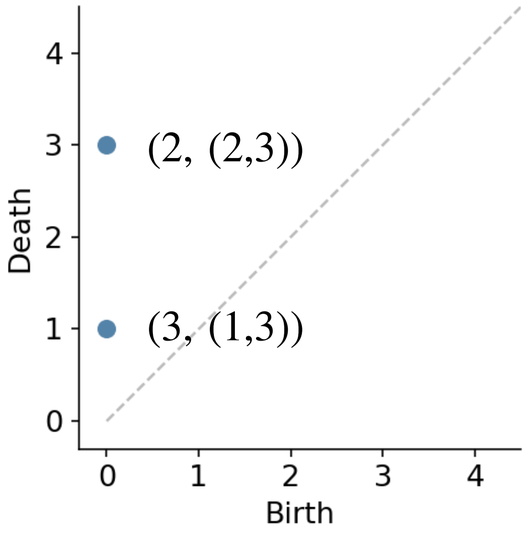


$\pi^X = ((1,2),\ (2,3))$

$\pi^Z = ((1,3),\ (2,3))$

Only finite bars





Barcodes from the distance matrices and the matchings

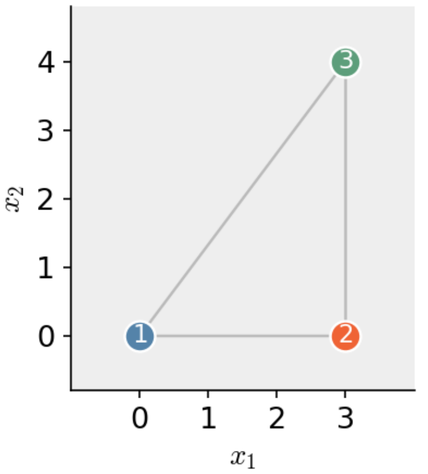
Pairwise distance matrices

$$A^X = \begin{pmatrix} 0 & 3 & 5 \\ 3 & 0 & 4 \\ 5 & 4 & 0 \end{pmatrix}, \qquad A^Z = \begin{pmatrix} 0 & 4 & 1 \\ 4 & 0 & 3 \\ 1 & 3 & 0 \end{pmatrix}$$

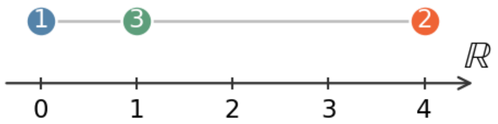
Matchings

$$\pi^X = ((1,2),\ (2,3)), \qquad \pi^Z = ((1,3),\ (2,3))$$

X



Z



In this setting the length of the bars can be deduced selecting the elements signalled by the matching from the distance matrix

$$A^X[\pi^X] = (3,4), \qquad A^Z[\pi^Z] = (1,3)$$

Comparing matchings of the input and the latent representation

Pairwise distance matrices

$$A^X = \begin{pmatrix} 0 & 3 & 5 \\ 3 & 0 & 4 \\ 5 & 4 & 0 \end{pmatrix}, \qquad A^Z = \begin{pmatrix} 0 & 4 & 1 \\ 4 & 0 & 3 \\ 1 & 3 & 0 \end{pmatrix}$$

Matchings

$$\pi^X = ((1,2),\ (2,3)), \qquad \pi^Z = ((1,3),\ (2,3))$$

Input to latent loss

$$A^X[\pi^X] = (3,4), \qquad A^Z[\pi^X] = (4,3).$$

$$\begin{aligned} \mathcal{L}_{X\to Z} &= \frac{1}{2}\left\|A^X[\pi^X] - A^Z[\pi^X]\right\|^2 \\ &= \frac{1}{2}\left[(3-4)^2 + (4-3)^2\right] \\ &= \boxed{1}. \end{aligned}$$

Latent to input loss

$$A^Z[\pi^Z] = (1,3), \qquad A^X[\pi^Z] = (5,4).$$

$$\begin{aligned} \mathcal{L}_{Z\to X} &= \frac{1}{2}\left\|A^Z[\pi^Z] - A^X[\pi^Z]\right\|^2 \\ &= \frac{1}{2}\left[(1-5)^2 + (3-4)^2\right] \\ &= \boxed{8.5}. \end{aligned}$$

A topological loss for auto encoders: summary

- Let X be a batch of input data, $Z = f_\theta(X)$ its latent representation (both are point clouds)
- Compute Vietoris—Rips persistent homology diagrams for both point clouds, giving:

$$(\mathrm{PD}^X, \pi^X), \qquad (\mathrm{PD}^Z, \pi^Z)$$

Persistence diagram

Matching between bars and critical cells

- You can obtain the bars by reading the distance matrices for the longest edges in each of the simplices

$$\mathrm{PD}^X \sim \mathbf{A}^X[\pi^X], \qquad \mathrm{PD}^Z \sim \mathbf{A}^Z[\pi^Z]$$

Matrix of pairwise distances

- Because both point clouds have the same number of points we can also select distances from X (resp. Z) using the matchings π^X (resp. π^Z)

$$\mathcal{L}_{X\to Z} = \frac{1}{2}\left\|\mathbf{A}^X[\pi^X] - \mathbf{A}^Z[\pi^X]\right\|^2, \qquad \mathcal{L}_{Z\to X} = \frac{1}{2}\left\|\mathbf{A}^Z[\pi^Z] - \mathbf{A}^X[\pi^Z]\right\|^2$$

Moor, M., Horn, M., Rieck, B., and Borgwardt, K. (2020). Topological autoencoders. In *International conference on machine learning* (pp. 7045-7054). PMLR.

An application to synthetic data

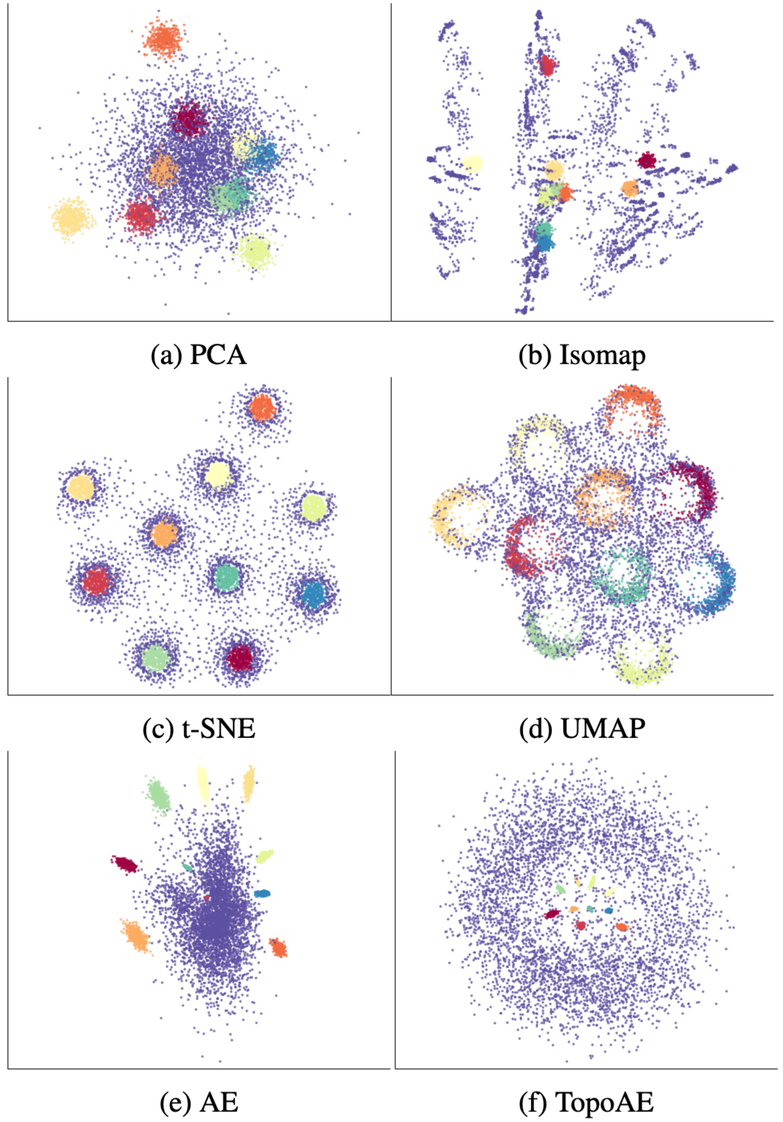
- Create a synthetic dataset of spheres of dimension 100 in $\mathbb{R}^{101}$
- 10 spheres of radius $r = 5$, and random small displacements of their centers to the origin
- 1 extra sphere of radius $5r$ enclosing all of them
- Use point clouds from this model to train a topological auto encoder, and compare the latent representation of a point cloud to common dimensionality reduction techniques

An application to synthetic data

Figure from: Moor et *al.* (2020)

The topological auto encoder is the method that best captures the nesting structure of the data!

The authors also present embedding quality measures for MNIST, FMNIST and CIFAR that favor the method



To sum up

Today we have learnt how to...

- Learn the best vectorisations for a given task
- Learn the filtration by optimising features in the barcode
- Learn how to apply this topological loss to an autoencoder

Next time we will see how to use persistence for interpretability!

Thank you for your attention!



AIDOS LAB
AI FOR DATA-ORIENTED SCIENCE

UNI
FR

Lecture 9 — Topological losses

Inés García Redondo — November 10, 2026